Knowledge-Based Systems 259 (2023) 110090

Contents lists available at ScienceDirect

Knowledge-Based Systems

journal homepage: www.elsevier.com/locate/knosys

A sinusoidal social learning swarm optimizer for large-scale optimization

Nengxian Liu [a], Jeng-Shyang Pan [a,b,c,*], Shu-Chuan Chu [b], Pei Hu [b]

[a] College of Mathematics and Computer Science, Fuzhou University, Fuzhou, China
[b] College of Computer Science and Engineering, Shandong University of Science and Technology, Qingdao, China
[c] Department of Information Management, Chaoyang University of Technology, Taichung 41349, Taiwan

ARTICLE INFO

Article history:
Received 1 May 2021
Received in revised form 31 October 2022
Accepted 1 November 2022
Available online 5 November 2022

Keywords:
Sinusoidal function
Particle swarm optimization (PSO)
Trapezoidal population size reduction
Large-scale optimization problems

ABSTRACT

Large-scale optimization problems are much more difficult compared to traditional optimization problems because they have a larger search space and more numerous local optimum. This paper presents a sinusoidal social learning swarm optimizer (SinSLSO) to effectively tackle large-scale optimization problems. In SinSLSO, sinusoidal function is employed to dynamically adjust the learning probability of particles in the population to balance exploration and exploitation capabilities. Meanwhile, the trapezoidal population size reduction strategy is utilized to make a trade-off between the diversity and convergence speed of SinSLSO. In addition, a new learning strategy is designed to prevent SinSLSO from trapping into a local optimum. Experiments are carried out on two widely used sets of large-scale benchmark functions (i.e., CEC2010 and CEC2013) and the SinSLSO is compared with eleven state-of-the-art algorithms. What is more, the proposed SinSLSO is applied to feature selection problem. The comparison results illustrate the competitive performance of SinSLSO in terms of the quality on most of test problems.

1. Introduction

Optimization is a critical research field in science and engineering since it usually exists in many practical applications [1,2]. For example, cloud workflow scheduling [3], deployment in wireless sensor networks [4], task assignment problems [5] and others [6–8]. In the past few decades, to address these complex problems, researchers have proposed many meta-heuristic algorithms such as differential evolution (DE) [9–11], monkey king evolution [12,13], particle swarm optimization (PSO) [14], quasi-affine transformation evolutionary [15], fish migration optimization [16] and artificial bee colony (ABC) [17]. These meta-heuristic algorithms have been proven to have many advantages in solving numerical optimization and practical engineering application problems, and have attracted many researchers' attention. However, most of these algorithms are only effective in solving low-dimensional problems, and their performance will deteriorate rapidly when solving the problems with dimensions exceeding 100 [18–21], called large-scale optimization problems. And such phenomenon is known as "the curse of dimensionality". The reason is that as the size of the dimension grows, the search space and the number of local optima will increase exponentially [18,22]. Therefore, these bring great challenges to the search efficiency of current meta-heuristic algorithms.

Among these meta-heuristic algorithms, PSO is one of the most popular algorithms owing to its simplicity and effectiveness, and it was first proposed by Kennedy et al. in 1995 [14]. In standard PSO, each particle learns from the personal best information and the global best information of the entire population, which may cause the algorithm to get into stagnation or converge prematurely. On the other hand, PSO algorithm may easily fall into local region when solving complicated multi-modal problems with a large number of local optima. To enhance the effectiveness, many scholars took inspirations from nature and human society and proposed plenty of PSO variants. Inspired by social animal colony, Chen et al. [23] proposed a improved PSO algorithm having an aging leader and challengers, called ALC-PSO. Liang et al. [24] proposed an important PSO variant, called comprehensive learning PSO (CLPSO), in which each particle updates its dimensions by learning from different personal best positions. Based on CLPSO, researchers propose some other PSO variants such as heterogeneous comprehensive learning PSO [25] and biogeography-based learning PSO [26]. Zhan et al. [27] introduced

* Corresponding author at: College of Computer Science and Engineering, Shandong University of Science and Technology, Qingdao, China.
E-mail addresses: lylnx@fzu.edu.cn (N. Liu), jspan@cc.kuas.edu.tw (J.-S. Pan), scchu0803@gmail.com (S.-C. Chu), huxiaopei163@163.com (P. Hu).

https://doi.org/10.1016/j.knosys.2022.110090
0950-7051/

an orthogonal learning PSO (OLPSO) via orthogonal experimental design. Chen et al. [28] enhanced the overall performance of PSO by using an adaptive population size. Sun et al. [29] proposed a PSO variant using the fitness estimation strategy. Some recent research works on PSO are summarized as in Table 1.

Although many PSO variants have been proposed and achieved better performance than the canonical PSO, most of them are only designed for low-dimensional problems. To address relatively high-dimensional problems, Cheng et al. [30] designed a multi-swarm framework with a pairwise competition mechanism. Zhao et al. [31] introduced a dynamic multi-swarm PSO (DMS-L-PSO) for the large-scale optimization problem. Recently, some researchers have introduced several PSO variants to deal with large-scale optimization via designing novel learning strategies [18–22,32]. In general, these PSO variants can reserve high population diversity to improve the exploration capability. In the related work section, some of these variants will be explained in detail. Though these PSO variants show promising performance in solving large-scale optimization, premature convergence is still their main challenge. Meanwhile, in order to escape from the local optimum, effective diversity enhancement strategies would be designed to further enhance the population diversity.

On the other hand, there are a large number of large-scale optimization problems in the real world, which motivate us to propose new optimization algorithms. For example, the large-scale convex optimization problem with equality constraints in Internet of Things (IoT) [33], finding the multiple longest common subsequences (MLCS) among many long sequences (i.e., the large scale MLCS problem) [34], large-scale crossing waypoints locating in air route network [35], large-scale combined heat and power economic dispatch problems [36], and large-scale flexible job shop scheduling problem [37].

Motivated by the needs of real-world, and to alleviate the problems of premature convergence and falling into local optimum, in this study, a sinusoidal social learning swarm optimizer (SinSLSO) is proposed to effectively tackle the large-scale optimization problems. First, to make a trade-off between exploration and exploitation capabilities of SinSLSO, sinusoidal function is adopted to dynamically adjust the learning probability of particles in the population. Second, SinSLSO adjusts the population size in a trapezoidal manner to obtain a balance between the population diversity and convergence speed. Finally, a novel particle learning strategy is introduced in SinSLSO to prevent it from trapping into a local optimum. Two commonly used benchmark sets of large-scale problems (i.e., CEC2010 and CEC2013) are adopted to verify the performance of the SinSLSO. The extensive comparison results with several state-of-the-art methods indicate the effectiveness of our proposed SinSLSO.

The rest of this paper is structured as follows. A brief introduction about the canonical PSO algorithm and the approaches for dealing with the large-scale optimization problems are given in Section 2. Section 3 describes the proposed SinSLSO algorithm in detail. In Section 4, extensive experiments are carried out to evaluate the effectiveness and efficiency of SinSLSO through comparison with several state-of-the-art algorithms. Finally, the conclusion of this paper is summarized in the last Section.

2. Related work

2.1. Particle swarm optimization (PSO)

PSO is one of population-based search algorithm, which simulates the predation of birds or fish. In PSO, each particle owns its position and velocity, which can be updated via using the equations as follows.

$$x_{id}(t+1) = x_{id}(t) + v_{id}(t+1) \tag{1}$$

$$v_{id}(t+1) = w v_{id}(t) + c_1 r_1 (pbest_{id}(t) - x_{id}(t)) + c_2 r_2 (gbest_{id}(t) - x_{id}(t)) \tag{2}$$

where $\boldsymbol{X_i}(t) = (x_{i1}(t), x_{i2}(t), \ldots, x_{iD}(t))$ and $\boldsymbol{V_i}(t) = (v_{i1}(t), v_{i2}(t), \ldots, v_{iD}(t))$ denote the position and velocity of particle i at tth generation, respectively, where $i = 1, 2, \ldots, ps$, ps means the population size and D represents the dimensionality of the problem. $\boldsymbol{pbest_i}(t) = (pbest_{i1}(t), pbest_{i2}(t), \ldots, pbest_{iD}(t))$ and $\boldsymbol{gbest_i}(t) = (gbest_{i1}(t), gbest_{i2}(t), \ldots, gbest_{iD}(t))$ are the best history position of particle i and the whole population, and they are called the personal best position and the global best position, respectively. w is the inertia weight to control the influence of historical velocity. Both c_1 and c_2 are cognitive parameters which pushes the particle to its ***pbest*** and pulls the particle to the current ***gbest***, respectively, to balance the population diversity and convergence speed. Both r_1 and r_2 are random numbers uniformly generated within [0, 1]. Generally, the second and the third parts in the right side of Eq. (2) are called the cognitive part and the social part, respectively [14].

2.2. Meta-heuristic algorithms for the large-scale problems

Usually, the meta-heuristic algorithms designed for solving large-scale optimization problems can be grouped into two categories. The first category is cooperative coevolutionary meta-heuristic algorithms, and the other category is the meta-heuristic algorithms with novel and efficient updating strategies.

2.2.1. Cooperative coevolutionary algorithms (CCEAs)

The main idea of cooperative coevolution (CC) is divide-and-conquer, which partitions the large-scale problem into several small subproblems, and then optimizes each subproblem separately with the corresponding meta-heuristics. Since Potter [39] introduced the first CCEA (i.e. cooperative coevolution genetic algorithm, CCGA), a larger number of CCEAs have been proposed via integrating CC with different meta-heuristic algorithms [49–51]. In [49], Frans van den et al. first proposed the CCPSO-S_K by combining CC with PSO. CCPSO-S_K randomly decomposes the decision variables into D/K subcomponents with each having K variables and then optimizes each subcomponent with classical PSO separately. The performance of CCEAs depends largely on their grouping strategy. In general, different problems require different grouping strategies. Therefore, researchers pay attention to designing grouping strategies. Yang et al. [52] presented a random grouping strategy via randomly dividing the D-dimensional problem into m S-dimensional ($S \ll D$) subproblems at each generation. The authors of [52] proposed the DECC-G by combining this random strategy with DE and DECC-G has well performance on several 1000-D problems. Following DECC-G, they developed another CC algorithm (multilevel cooperative coevolution, called MLCC), in which the value of S is changed dynamically during each evolutionary cycle [53]. Li et al. [50] introduced a cooperatively coevolving PSO (CCPSO2) which also uses a group size pool.

Omidvar et al. [54] proposed another type of grouping strategy which can detect the interdependent decision variables, named differential grouping (DG). Since DG can obtain reasonable performance via detecting interdependent variables, several its variants, such as XDG [55], GDG [56] and DG2 [57], have been further introduced. DG and its variants usually can partition variables into groups more accurately than the random grouping strategy. Although CCEAs have good performance in large-scale optimization, they suffer from two limitations, which limit their wide application. On one hand, the performance of CCEAs is heavily sensitive to the grouping strategy, and it needs plenty of function evaluations (FEs) for a good decomposer to detect the interdependency among variables. On the other hand, a good CCEA often costs a larger number of FEs, especially when the number of subcomponents is large.

Table 1
Some recent research works on PSO.

Ref.	Brief explanation
Liu et al. [38]	This paper proposed an improved particle swarm optimization based on an energy management strategy for optimal sizing and configuration of standalone photovoltaic scheme components.
Ayyash et al. [39]	The main aim of this paper is hybridization of the particle swarm optimization (PSO) and gravitational search algorithm (GSA) to optimize the performance of earthquake energy dissipation systems (i.e., damper devices) simultaneously with optimizing the characteristics of the structure.
Rashid et al. [40]	This study proposed a hybrid particle swarm optimization algorithm with linearly decreasing inertia weight for the optimization of analog circuit design.
Sato et al. [41]	This paper proposed total optimization of energy networks in a smart city (SC) via multi-swarm differential evolutionary particle swarm optimization (MS-DEEPSO).
Li et al. [42]	This study integrated the simulated annealing strategy into the improved PSO to improve the ability of particles jumping out of the local optimum, and an innovative approach for generating a better covering array was proposed.
Pozna et al. [43]	This paper presented a hybrid metaheuristic optimization algorithm that combines Particle Filter (PF) and Particle Swarm Optimization (PSO) algorithms and applied the proposed algorithm to the optimal tuning of Proportional–Integral-fuzzy controllers.
Lin et al. [44]	This paper proposed an improved multiobjective adaptive particle swarm optimization (MOAPSO), which has the characteristics of faster convergence, higher efficiency and low computational complexity. The proposed MOAPSO was applied to the analysis and optimization of urban public transport lines.
Luo et al. [45]	This work proposed an image encryption scheme based on the updating processes of particle swarm optimization algorithm and hyperchaotic system.
Van et al. [46]	This work proposed the novel two-phase metaheuristic method that combines the distinctive features provided by evolutionary structural optimization (ESO) and comprehensive learning particle swarm optimization (CLPSO) for the design of steel structures with standard sections.
Zhu et al. [47]	This paper proposed an improved particle swarm optimization algorithm based on elastic collision strategy (EC-PSO) to optimize the design of DNA sequences by using an adaptation function that satisfies multiple constraints.
Seck–Tuoh–Mora et al. [48]	To deal with permutation flow shop scheduling problem, this paper proposed an improvement of PSO with a simple adaptive local search strategy based on the previous Alternate Two-Phase PSO (ATPPSO) method and the neighborhood concepts of the Cellular PSO algorithm proposed for continuous problems.

$$v_{id}(t+1) = \begin{cases} r_1 v_{id}(t) + r_2(x_{kd}(t) - x_{id}(t)) + \varphi r_3(\overline{x}_d(t) - x_{id}(t)) & \text{if } p_i(t) \le PL_i \\ v_{id}(t) \quad \text{otherwise} \end{cases} \tag{3}$$

Box I.

2.2.2. Meta-heuristic algorithms with novel learning strategies

Different to the CCEAs, some researchers focus on designing novel learning or updating strategies to aid meta-heuristic algorithms to address large-scale optimization problems. The reason is that the proposed updating strategies can reserve the diversity of the population, and enable meta-heuristic algorithms to have good the exploration capability. Enlightened from the social learning behavior of social animals, Cheng et al. proposed a novel social learning PSO (SL-PSO) [18]. Different from the classical PSO, firstly, all particles in SL-PSO are sorted based on their fitness values. And then each particle i ($1 \le i < ps$) learns from demonstrators with better fitness values in the current population, and learns from the mean position of the entire population. The velocity updating formula of SL-PSO is given as in Eq. (3) (see Box I), where k is the index of the demonstrator for the d dimension of particle i, k can be the index of any better particle, ($i < k \le ps$). r_i ($i = 1, 2, 3$) and $p_i(t)$ are the random numbers within [0, 1]. φ is a controlling parameter for $\overline{\boldsymbol{X}}(\boldsymbol{t})$, and $\overline{\boldsymbol{X}}(\boldsymbol{t})$ is the mean position of the entire population. PL_i is the learning probability. Note that SL-PSO does not use the personal best position and global best position to guide the learning.

Inspired from the competition of human society, competitive swarm optimizer (CSO) [18] was proposed by Cheng and Jin, which adopts one predominant particle and mean position of the entire swarm instead of personal best and global best positions. Following the CSO, Yang et al. [58] developed the segment-based predominant learning swarm optimizer (SPLSO), which divided the D dimensions into a number of segments. Other algorithms similar to SL-PSO and CSO include LLSO [19], MSL-PSO [20], TPLSO [21] and RBLSO [32]. Further observing on these algorithms, it can be found that the learning or updating strategies of these optimizers highly enhance the diversity of the population, therefore, premature convergence may be prevented. Although the large amount of works exist in coping with large-scale optimization, both premature convergence and dropping into local optimum are still the primary challenges in solving large-scale optimization.

3. Sinusoidal social learning swarm optimizer

3.1. Motivation

The balance between the population diversity and convergence is significant important for meta-heuristic algorithms to cope with large-scale optimization problems [20,58], Since when the dimension of the problem increases the search space of it will increase exponentially, and the problem will become more and more challenges for meta-heuristic algorithms to optimize. PSO has the advantages of easy implementation and fast convergence. However, only using personal best information and global best particle to direct the population evolution, the population diversity will be lost rapidly after several generations. Hence, the original PSO is not suitable for solving large-scale problems. In the literature, we can see that some improved PSO variants are presented to solve large-scale problems by designing new learning strategies, such as CSO, SL-PSO, LLSO, RBLSO. They adopt

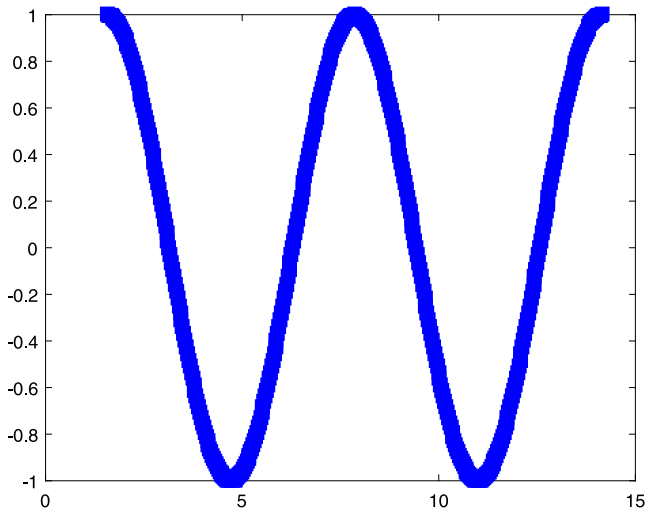

Fig. 1. The sinusoidal function.

one (a few) predominant particle (particles) instead of *pbest* and *gbest* to update particles, and they are prone to adopting worse particles to implement learning. Moreover, to avoid premature convergence, the update or learning methods of these optimizers greatly increase the diversity of the swarm. However, some of them do not have a good trade-off between the population diversity and convergence [21]. Generally, the balance between the population diversity and convergence of meta-heuristic algorithms corresponds to make a compromise between exploration and exploitation capabilities. If the algorithm has a strong exploration ability, it usually has a good population diversity, and vice versa.

To further enhance the effectiveness of meta-heuristic algorithms to handle large-scale optimization problems, we get inspiration from simple mathematical functions and human society. Different to the SL-PSO, which uses the reduced learning rate to set the learning probability of each particle after sorting the population according to fitness values, we adopt the sinusoidal function (shown in Fig. 1) to adjust the learning probability of each particle in our proposed SinSLSO. SL-PSO with a reduced learning rate only benefits to the particles with poor fitness values and concentrates on the exploration ability. Whereas SinSLSO with the sinusoidal function can balance between the exploration and exploitation capabilities. Because the sinusoidal function can change the direction of learning probability of particles: from increasing to decreasing and vice versa. Therefore, in the exploration step, particles with worse fitness values have higher learning probability so more worse particles can evolve to focus on exploration. In the exploitation step, particles with better fitness values have higher learning probability so more better particles can evolve to focus on exploitation.

Meanwhile, population size plays a significantly important role in meta-heuristic algorithms, and the robustness as well as effectiveness of algorithm are also affected by it [28,59]. A small population size tends to have faster convergence, but it increases the risk of convergence to a local optima. On the other hand, the large population size can promote exploration ability of the algorithm, but the convergence speed of algorithm tends to slow down. Therefore, an appropriate population size is essential to the effectiveness of the algorithm. Generally, the population needs higher diversity in the early stage of population evolution, so a larger population is required. On the contrary, the algorithm needs a faster convergence speed in the later stage of population evolution, so a smaller population is required. Therefore, we use a trapezoidal function to reduce the population size of our proposed SinSLSO to make a compromise between the diversity and convergence.

Moreover, inspired by the social learning and recent proposed algorithms such as SL-PSO and CSO, we observe that these recent algorithms do not use the ***pbest*** and ***gbest*** to guide the evolution of population. On the contrary, they often directly utilize predominate particles in the current population and the mean position of the current population to update particles and present potentials in addressing large-scale problems owing to the improved diversity. Enlightened by these ideas, we design a new learning strategy to update the velocity of the particle to avoid trapping into local optima.

3.2. SinSLSO

In this subsection, a sinusoidal social learning swarm optimizer (SinSLSO) for large-scale optimization is described in detail. The overall framework of our proposed SinSLSO is illustrated in Fig. 2. In Fig. 2, the red slashes are drawn by the sinusoidal function to stand for the learning probability, and the green circles denote that the corresponding particles are updated. The SinSLSO algorithm chiefly includes four parts: initialization, exploration step, exploitation step, and population reduction.

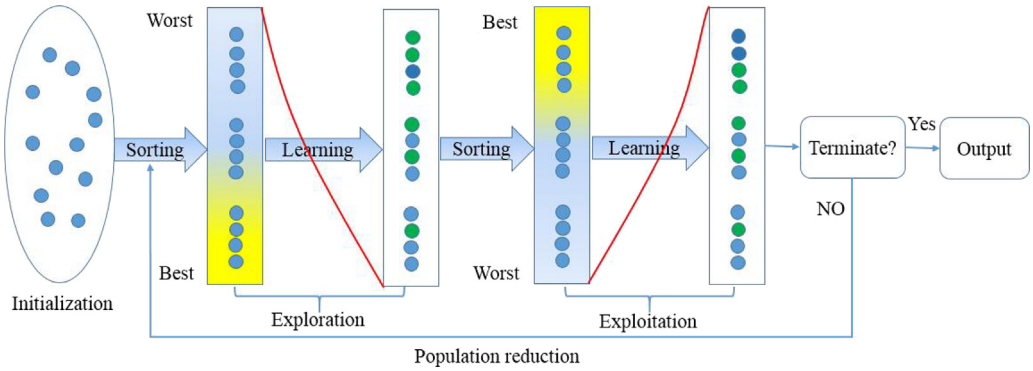

Fig. 2. Framework of the proposed SinSLSO. In the exploration step, the swarm is firstly sorted from the worst to the best based on the fitness values. Then, if the randomly generated number between (0, 1) is less than the learning probability (generated by the sinusoidal function), the particle will learn from the demonstrators with better fitness values. In the exploitation step, the swarm is firstly sorted from the best to the worst according to the fitness values. Then, if the randomly generated number between (0, 1) is less than the learning probability (generated by the sinusoidal function), the particle will learn from the demonstrators having better fitness values. The red slashes are drawn by the sin function to represent the learning probability, and the green circles indicate that the corresponding particles are updated. After exploration and exploitation steps, the population size will decrease in a trapezoidal manner.

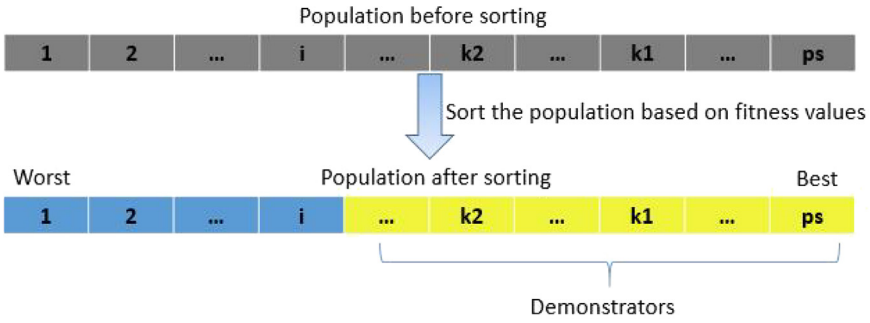

Fig. 3. Population sorting and exploration learning in SinSLSO.

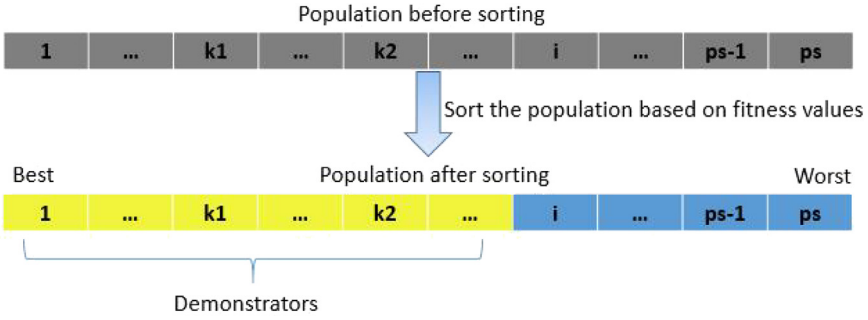

Fig. 4. Population sorting and exploitation learning in SinSLSO.

$$v_{id}(t+1) = \begin{cases} r_1 v_{id}(t) + r_2(x_{k_1 d}(t) - x_{id}(t)) + \varphi r_3(x_{k_2 d}(t) - x_{id}(t)) & \text{if } p_i(t) \le PL_i \\ v_{id}(t) & \text{otherwise} \end{cases} \quad (5)$$

Box II.

Firstly, we randomly initialize the population and evaluate their fitness values.

Secondly, the population enters into exploration step. In this step, the population is firstly sorted from worst to best based on their fitness values. Then the sinusoidal function is used to adjust the learning probability (PL) of particles, which is given below.

$$PL_i = 1/2 \times (\sin(\pi/2 + i/ps \times \pi) + 1) \quad (4)$$

where i is the index of the particle. In this step, the poorer particles have a higher learning probability and will explore more solution space, which is beneficial to the exploration of the population. A particle will directly enter into the next population if a randomly generated number between (0, 1) is greater than its learning probability. Otherwise, the particle will update its velocity according to a new designed learning strategy. The equation for updating the velocity is given as in Eq. (5) (see Box II), where k_1, k_2 are the index of the demonstrators of particle i. It is noted that ($k_1 > k_2 > i$), and k_1 and k_2 are generated independently for each dimension d, as shown in Fig. 3. Since there are not enough demonstrators for the particles $ps - 1$ and ps, they will directly enter into the next generation. φ is a random number within $[\varphi_{\max}, \varphi_{\min}]$, which is generated according to the following equation. This random strategy is good for searching the solution space and jumping out of the local optima regions [60,61].

$$\varphi = \varphi_{\min} + rand() \times (\varphi_{\max} - \varphi_{\min}) \quad (6)$$

Thirdly, the population enters into exploitation step. In this step, we firstly sort the population from best to worst according to their fitness values, and then adopt the sinusoidal function to adjust the learning probability of particles, which is shown as follows.

$$PL_i = 1/2 \times (\sin(3/2 \times \pi + i/ps \times \pi) + 1) \quad (7)$$

where i is the index of the particle. Different to exploration step, in this step, the better particles have a higher learning probability and will search the solutions in promising regions, which gives great benefit to the exploitation of the population. Particles in the step also use the former learning strategy Eq. (5), except ($k_1 < k_2 < i$), and k_1 and k_2 are also generated independently for each dimension d, as given in Fig. 4.

Fourthly, the algorithm adjusts the population size (ps) according to the trapezoidal function, which is formulated as in Eq. (9) (see Box III), where NFE is the number of function evaluations, NFE_{tp} is the turning point of the number of function evaluations, at which the ps begins to reduce. $ps_{\max}$ and $ps_{\min}$ are maximum and minimum of ps, respectively. An example is given in Fig. 5, where $ps_{\max}$, $ps_{\min}$, NFE_{tp}, and $NFE_{\max}$ are set to 800, 400, $E6$ and $3E6$, respectively. At the beginning of evolution, a larger population size is good for fully exploring the solution space and preserving the diversity of population. Whereas, at the middle and later stages of evolution, relative smaller population size gives benefit to a faster convergence speed. In the process of decreasing population size, some of the particles except the best one will be randomly deleted in order to preserve population diversity. Therefore, this reduction strategy can well balance the diversity and convergence speed.

$$ps = \begin{cases} ps_{\max} & if \quad NFE < NFE_{tp} \\ ps_{\max} + (ps_{\min} - ps_{\max}) * (NFE - ps)/(NFE_{\max} - ps) & otherwise \end{cases} \tag{8}$$

Box III.

Steps 2–4 of the procedure will be repeated until the stop condition is met (generally, the maximum number of function evaluations is exhausted).

Algorithm 1 describes the pseudo code of SinSLSO.

Algorithm 1 The pseudocode of the proposed SinSLSO

Input: population size $ps_{\max}$, $ps_{\min}$, ps, dimension of the problem D, learning rate $\varphi_{\max}$, $\varphi_{\min}$, φ, $MaxNFE = 3 \times 10^6$, $NFE = 0$, $gen = 0$, $Type = 1$.

Output: The best solution X^* and its fitness value $f(X^*)$.

Randomly initialize $\boldsymbol{P}$, and evaluate the fitness values **F** of particles in the $\boldsymbol{P}$, get the best solution: X^*, $NFE = ps$
while $(NFE < MaxNFE)$ **do**
  $\varphi = \varphi_{\min} + rand() \times (\varphi_{\max} - \varphi_{\min})$ // Randomly generate a parameter φ between $\varphi_{\min}$ and $\varphi_{\max}$
  if $Type = 1$ **then** //Generate learning probability for exploration
    for $i = 1$ to ps **do**
      $PL_i = 1/2 \times (\sin(\pi/2 + i/ps \times \pi) + 1)$
    end for
  elseif $Type = 2$ **then**//Generate learning probability for exploitation
    for $i = 1$ to ps **do**
      $PL_i = 1/2 \times (\sin(3/2 \times \pi + i/ps \times \pi) + 1)$
    end for
  end if
  Sort particles in the $\boldsymbol{P}$ according to the fitness values **F**
  for $i = 1$ to $ps - 2$ **do**
    $P_i = randr(0, 1)$ // $randr(0, 1)$ randomly produce a real value between 0 and 1
    if $P_i \leq PL_i$ **then**
      for $j = 1$ to D **do**
        $k_1 = randn(i + 1, ps)$ // $randn(i + 1, ps)$ randomly produces an integer between $i + 1$ and ps
        $k_2 = randn(i + 1, ps)$ // $randn(i + 1, ps)$ randomly produces an integer between $i + 1$ and ps
        if $k_1 < k_2$ *and* $Type = 1$ **then** swap(k_1, k_2) **end if**
        if $k_1 > k_2$ *and* $Type = 2$ **then** swap(k_1, k_2) **end if**
        Update the j_{th} dimension of velocity according to Eq. (5)
        Update the j_{th} dimension of position according to Eq. (1)
      end for
      Evaluate the fitness value of the particle X_i
      $NFE = NFE + 1$
    end if
  end for
  Update the best solution: X^*
  $gen = gen + 1$
  Calculate $ps(gen + 1)$ according to Eq. (8)
  if $ps(gen + 1) < ps(gen)$ **then**
    randomly delete $ps(gen) - ps(gen + 1)$ particles from the $\boldsymbol{P}$ expect the best particle
  end if
  Change the value of $Type$
end while

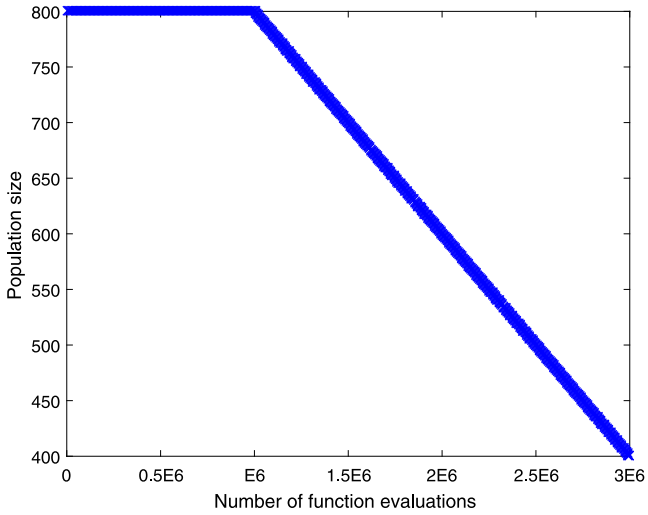

Fig. 5. Population size reduction using the trapezoidal manner.

3.3. Difference between SinSLSO and some variants of PSO

According to the description in the previous section, we can find that SinSLSO contains three new properties: using sinusoidal function to make a trade-off between exploration and exploitation, a new learning strategy, and a new population size reduction mechanism. These unique properties distinguish SinSLSO from the other current PSO variants. Their differences are given as follows.

(1) Difference in exploration and exploitation. Compared with CSO, CSO only updates the inferior particles who fail in the competition in each generation, so it focuses more on exploration. Similar to CSO, the inferior particles of SL-PSO have more chances to evolve because SL-PSO always sets the inferior particles with a higher learning probability than the superior ones. Hence, SL-PSO also concentrates more on exploration. With regard to the DLLSO, DLLSO introduces a level learning mechanism to make a balance between the exploration and exploitation. In DLLSO, the particles in the lower level focus on exploration, while the ones in higher level pay attention to exploitation. However, the balance of this level learning mechanism mainly depends on the level size. As for TPLSO, TPLSO employs two strategies mass learning and elite learning. In the mass learning, the winner in the group will enter the next generation directly, while the inferior and the worst particles evolve by learning from the winner, so the mass learning focus on exploration. In the elite learning, the top $ps/2$ particles will learn from the better particles, and the rest particles will pass to the next generation directly. Thus, the elite learning focus on exploitation.

Our proposed SinSLSO adopts sinusoidal function to adjust the learning probability of particles. At the exploration step, particles with worse fitness values have higher learning probability so more worse particles can evolve to concentrate on exploration. At the exploitation step, particles with better fitness values have higher learning probability so more better particles can evolve to concentrate on exploitation.

(2) Difference in learning strategy. Different to the classical PSO, most of the recent proposed PSO variants for large scale optimization problems do not learn from the ***pbest*** and ***gbest***. In CSO, the learning strategy is that loser particle learns from the winner and the mean position of the population. The control parameter φ of CSO is set differently for the separate functions and inseparable functions. The learning strategy for SL-PSO is that each dimension of the particle learns from a demonstrator, which is independently generated for each dimension, and the mean position of the population. The control parameter φ of SL-PSO is set according to the dimension and user defined parameters. As for DLLSO, the learning strategy is that two superior particles in current population is selected as the exemplars for guiding the learning of inferior particles. The control parameter φ of DLLSO is set to 0.4 for all types of functions. With regard to the TPLSO, there are two learning strategies in the mass learning and one learning strategy in the elite learning. For detailed mass learning strategies, please refer to [21]. The elite learning strategy is that each particle learns from two different better particles in current population. The control parameter φ of TPLSO is also set differently for the separate functions and inseparable functions.

For our proposed SinSLSO, each dimension of the particle learns from two different demonstrators but it does not learn from the mean position, which is more benefit to reserve the population diversity. Another difference is that the control parameter φ of SinSLSO is set in a random way for all types of functions.

(3) Difference in population size. The population size has an important impact on the population diversity and convergence speed of the algorithm. For large-scale optimization problems, most of the recently proposed PSO variables set the population size ps to a relatively large number to maintain population diversity. For example, for a 1000D problem, CSO and LLSO set the population size to 500, and TPLSO sets the population size to 600. Whereas, the proposed SinSLSO uses the trapezoidal function to adjust the population size to balance the diversity and convergence speed.

3.4. Time complexity of the SinSLSO

In this study, the time complexity of the SinSLSO mainly considers the time to perform extra operation in each generation, except the time of function evaluations [18,19,21]. The reasons are that algorithms are usually compared under a fixed number of function evaluations, and the time cost of the function evaluation depends on the optimization problem. From the algorithm 1, we can find that it needs $O(ps + ps\log(ps))$ to set the learning probabilities of the particles and to sort the population (lines 4–13). In the worst case, the time to update all particles in the population is $O(ps \times D)$ (lines 14–28). In addition, it takes $O(ps)$ to adjust the population size (lines 31–34). Hence, the time complexity of the SinSLSO is $O(2 \times ps + ps\log(ps) + ps \times D)$. According to [19], LLSO needs $O(ps + ps\log(ps) + ps \times D)$ in each generation. So in comparison to LLSO, SinSLSO will need more time in each generation.

4. Theoretical analysis and convergence proof

To better understand the search mechanism of SinSLSO, we investigate the exploration and exploitation abilities of SinSLSO by theoretically comparing it with the LLSO [19] and the global PSO (GPSO) [14]. What is more, a convergence proof of SinSLSO will be given, which indicates that the SinSLSO, like the GPSO, will converge to equilibrium.

4.1. Theoretical analysis

(1) Exploration: Exploration plays an important role when the particles explore the search areas. For an EA, enhancing the exploration ability is to promote the diversity of the population, so that it can avoid dropping into the local areas and find the more promising areas. Especially, the exploration ability is considerably important for EAs to tackle multimodal problems with a large number of local areas. To study the exploration ability of SinSLSO, we rewrite the first equation of Eq. (5) as follows.

$$v_{id}(t+1) = r_1 v_{id}(t) + \theta_1(p_1 - x_{id}(t)) \tag{9}$$

$$\theta_1 = r_2 + \varphi r_3 \tag{10}$$

$$p_1 = \frac{r_2}{r_2 + \varphi r_3} x_{k_1 d}(t) + \frac{\varphi r_3}{r_2 + \varphi r_3} x_{k_2 d}(t) \tag{11}$$

According to the theoretical analysis of LLSO [19], the velocity update formulas of GPSO and LLSO can be rewritten to Eqs. (12) and (15) respectively.

$$v_{id}(t+1) = \omega v_{id}(t) + \theta_2(p_2 - x_{id}(t)) \tag{12}$$

$$\theta_2 = c_1 r_1 + c_2 r_2 \tag{13}$$

$$p_2 = \frac{c_1 r_1}{c_1 r_1 + c_2 r_2} pbest_{id}(t) + \frac{c_2 r_2}{c_1 r_1 + c_2 r_2} gbest_d(t) \tag{14}$$

$$v_{id}(t+1) = r_1 v_{id}(t) + \theta_3(p_3 - x_{id}(t)) \tag{15}$$

$$\theta_3 = r_2 + \phi r_3 \tag{16}$$

$$p_3 = \frac{r_2}{r_2 + \phi r_3} x_{rl_1, h_1, d}(t) + \frac{\phi r_3}{r_2 + \phi r_3} x_{rl_2, h_2, d}(t) \tag{17}$$

From the Eqs. (9), (12) and (15), we can observe that the difference among $p_i (i = 1, 2, 3)$ and the different guiding particles provide the main source of diversity. Comparing Eq. (11) with Eq. (17), we can conclude that SinSLSO has the same excellent exploration ability as LLSO because the indices k_1 and k_2 of Eq. (11) are similar to the indices rl_1, h_1 and rl_2, h_2 of Eq. (17). Compared to Eq. (14), we can observe that Eq. (11) has better chance to generate a higher degree of diversity. Because $x_{k_1 d}$ and $x_{k_2 d}$ of Eq. (11) are randomly selected for each particle. However, in Eq. (14), the $pbest_i$ is deterministically updated and always used by particle i and $gbest$ is also deterministically updated and shared by all particles [22].

(2) Exploitation: Exploitation is necessary when the particles exploit the search space. For an EA, enhancing the exploitation ability is to fully exploit the found promising areas quickly, so that better solutions can be found as fast as possible. Especially, the exploitation ability is considerably important for EAs to handle unimodal problems. In general, the difference between the exemplar and the updated particle is used to represent the exploitation ability [22].

To analyze the exploitation behavior of SinSLSO, for the ith particle X_i, two random exemplars X_{k1} and X_{k2} are selected from the better part of swarm. Then, we have

$$f(X_{k1}(t)) \le f(X_{k2}(t)) \le f(X_i(t)) \tag{18}$$

According to the definitions of $pbest$ and $gbest$ in the GPSO, we have the following formula:

$$\begin{cases} f(gbest(t)) \le f(pbest_i(t)) \le f(X_i(t)) \\ f(gbest(t)) \le f(pbest_{k1}(t)) \le f(X_{k1}(t)) \\ f(gbest(t)) \le f(pbest_{k2}(t)) \le f(X_{k2}(t)) \end{cases} \tag{19}$$

In the late stage of evolution, t will become very large and the following relationship holds.

$$gbest(t) \approx pbest_{k1}(t) \approx pbest_{k2}(t) \approx pbest_i(t) \tag{20}$$

Let

$$\begin{aligned} \Delta F_{GPSO}(t) &= |f(X_i(t)) - f(gbest)| \\ &= |f(X_i(t)) - f(\tfrac{gbest(t)+gbest(t)}{2})| \\ &\approx |f(X_i(t)) - f(\tfrac{gbest(t)+pbest_i(t)}{2})| \\ &= |f(X_i(t)) - f(p_2')| \end{aligned} \tag{21}$$

where p_2' is the expected value of p_2 in Eq. (14).

Similarly, for SinSLSO and LLSO, we have the following formula:

$$\begin{aligned} \Delta F_{SinSLSO}(t) &= |f(X_i(t)) - f(X_{k_1}(t))| \\ &= |f(X_i(t)) - f(p_1')| \end{aligned} \tag{22}$$

$$\begin{aligned} \Delta F_{LLSO}(t) &= |f(X_i(t)) - f(X_{rl_1,h_1}(t))| \\ &= |f(X_i(t)) - f(p_3')| \end{aligned} \tag{23}$$

where p_1' is the expected value of p_1 with $\varphi = 0$ in Eq. (11) and p_3' is the expected value of p_3 with $\phi = 0$ in Eq. (17). The expected value of $X_{k_1}(t)$ may equal to the expected value of $X_{rl_1,h_1}(t)$, so that we have $f(X_{k_1}(t)) \approx f(X_{rl_1,h_1}(t))$.

Combining the above formulas, we can derive the following relationship.

$$\Delta F_{SinSLSO}(t) \approx \Delta F_{LLSO}(t) \leq \Delta F_{GPSO}(t) \tag{24}$$

The above formula indicates that SinSLSO and LLSO have similar exploitation abilities, and they have a better ability than GPSO to exploit the small gaps between two positions whose fitness values are very similar.

4.2. Convergence proof

Similar to most previous theoretical convergence analyses of PSO [62,63,18,22], a deterministic implementation of our proposed SinSLSO is considered as theoretically analyze its convergence. Note that this proof does not guarantee a convergence to the global optimal value.

In general, the convergence of the entire population can be seen more specifically as the convergence of each dimension in any particle behavior vector. That is to say, convergence can be satisfied if and only if any dimension of the behavior vector of all particles is no longer changed. First, we take the update of $X_{i,j}(t)$ into account. $X_{i,j}(t)$ is updated using Eq. (25) if its learning probability satisfies $P_i \leq PL_i$.

$$\begin{aligned} v_{ij}(t+1) &= \tfrac{1}{2}v_{ij}(t) + \tfrac{1}{2}(x_{k_1 j}(t) - x_{ij}(t)) + \tfrac{1}{2}\varphi(x_{k_2 j}(t) - x_{ij}(t)) \\ x_{ij}(t+1) &= x_{ij}(t) + v_{ij}(t+1) \end{aligned} \tag{25}$$

where $\frac{1}{2}$ is the expected value of r_1, r_2 and r_3. Therefore, the convergence proof of our proposed SinSLSO can be equivalent to the convergence proof of the dynamic system depicted by Eq. (25).

Theorem 1. *The dynamic system depicted by Eq. (25) converges to an equilibrium.*

Proof. Let

$$\begin{aligned} \theta &= \tfrac{1+\varphi}{2}, \\ p &= \tfrac{1}{1+\varphi}x_{k_1 j}(t) + \tfrac{\varphi}{1+\varphi}x_{k_2 j}(t) \end{aligned} \tag{26}$$

then Eq. (25) can be simplified to:

$$\begin{aligned} v_{ij}(t+1) &= \tfrac{1}{2}v_{ij}(t) + \theta(p - x_{ij}(t)) \\ x_{ij}(t+1) &= x_{ij}(t) + v_{ij}(t+1) \end{aligned} \tag{27}$$

The search dynamics depicted in Eq. (27) can be regarded as a dynamical system, and the well-established theories of stability analysis in dynamical system can be used to analyze the convergence analysis of this system. To this end, we rewrite Eq. (27) as follows:

$$y(t+1) = Ay(t) + Bp \tag{28}$$

where

$$y(t) = \begin{bmatrix} v_{ij}(t) \\ x_{ij}(t) \end{bmatrix}, A = \begin{bmatrix} \frac{1}{2} & -\theta \\ \frac{1}{2} & 1-\theta \end{bmatrix}, B = \begin{bmatrix} \theta \\ \theta \end{bmatrix} \tag{29}$$

where A is named *state matrix* in dynamical system theory, p is named *external input* which drives the particle to a specific location and B is named *input matrix* which dominates external effect on the dynamics of the particle.

If there exists an equilibrium y^* which meets $y^*(t+1) = y^*(t)$ for any t, it can be computed from Eqs. (28) and (29):

$$y^* = \begin{bmatrix} 0 & p \end{bmatrix} \tag{30}$$

which indicates that all particles will finally stabilize at the same location, supposing that p is constant, i.e., a local or global optimum has been found, so that p will not be updated.

Convergence represents that all particles will eventually stabilize at the equilibrium point y^*. From the perspective of dynamical system theory, we can know that the convergence property relies on the eigenvalues of the state matrix A:

$$\lambda^2 - (\frac{3}{2} - \theta)\lambda + \frac{1}{2} = 0 \tag{31}$$

the eigenvalues of Eq. (31) are:

$$\begin{cases} \lambda_1 = \frac{3}{4} - \frac{\theta}{2} + \frac{\sqrt{(\frac{3}{2}-\theta)^2 - 2}}{2} \\ \lambda_2 = \frac{3}{4} - \frac{\theta}{2} - \frac{\sqrt{(\frac{3}{2}-\theta)^2 - 2}}{2} \end{cases} \tag{32}$$

The necessary and sufficient condition for convergence is that the equilibrium point is a stable attractor, i.e., $|\lambda_1| < 1$ and $|\lambda_2| < 1$, resulting in:

$$\theta > 0 \tag{33}$$

where if θ is substituted by φ with Eq. (26), the condition for convergence on φ is:

$$\varphi > -1 \tag{34}$$

Thus, $\varphi > 0$ is a sufficient condition for the convergence of the system.

5. Experimental results

To verify the efficiency and effectiveness of SinSLSO, a series of experiments are carried out on two widely used benchmark suites (i.e., CEC2010 [64] and CEC2013 [65]) from different perspectives. Functions in two test suites have different characteristics. The CEC2010 benchmark suite contains 20 functions, which can be categorized into three groups: separable, partially separable, and non-separable functions. The CEC2013 benchmark suite consists of 15 functions, which are based on CEC2010 and have several new characteristics, such as subcomponents with nonuniform sizes and overlapping functions. Therefore, compared with the functions in CEC2010, functions in the CEC2013 benchmark suite are more complicated and more challenging to optimize. Detailed information about these functions, readers can refer to [64,65].

The SinSLSO is compared with a series of recently proposed and state-of-the-art algorithms. Recently proposed algorithms

Table 2
Parameters settings.

Algorithm	Parameters settings
SinSLSO	$ps_{\max} = 800, ps_{\min} = 400, \varphi_{\max} = 0.3, \varphi_{\min} = 0.25, NFE_{\max} = 3 \times 10^6, NFE_{tp} = NFE_{\max}/3$
RLLPSO	$ps = 500, \varphi = 0.4$
TPLSO	$ps = 600, \varphi_1 = 0.15, \varphi_2 = 0.2, K = 3$
DLLSO	$ps = 500, \varphi = 0.4, S = \{4, 6, 8, 10, 20, 50\}$
RBLSO	$ps = 500, \varphi_1 = 0.1, \varphi_2 = 0.15$
CSO	$ps = 500, \varphi_1 = 0.1, \varphi_2 = 0.15$
SL-PSO	$ps = 200, M = 100, \beta = 0.001, \varphi = 0.1$
DMS-L-PSO	$ps = 900, \omega = 0.729, c_1 = c_2 = 1.49445, R = 10, L = 100, L\_FEs = 200$
CCPSO2	$ps = 30, S = \{2, 5, 10, 50, 100, 250\}$
DECC-DG	$ps = 50, \epsilon = 1e-3$
DECC-G	$ps = 100, S = 100$
MLCC	$ps = 100, \epsilon = 1e-4, S = \{5, 10, 25, 50, 100\}$

include RLLPSO [66], TPLSO [21], DLLSO [19], and RBLSO [32]. And state-of-the-art algorithms include CSO [22], SL-PSO [18], DMS-L-PSO [31], CCPSO2 [50], DECC-DG [54], DECC-G [52], and MLCC [53]. The results of TPLSO, DLLSO, and RBLSO are cited from their corresponding papers, the results of RLLPSO and DECC-DG are cited from [66], and the results of CCPSO2 are cited from [21]. The rest algorithms are conducted on a PC with two Intel(R) Core(TM) i5 3.3 GHz and 8.0 GB memory in Operating System Windows 7 with MATLAB 2016b. Each algorithm is independently run 20 times to obtain statistical results. The codes of CSO, SL-PSO, DMS-L-PSO, DECC-G and MLCC are provided by the original authors.

5.1. Parameter settings

The parameters of our proposed SinSLSO employed in the experiments are given as follows. For the sake of fairness, the maximum number of fitness evaluations is 3×10^6. During the period of population size reduction, the maximum and minimum number of population size ps are set to 800 and 400, respectively. The maximum and minimum value of parameter φ are set to 0.3 and 0.25 through experiment. The turning point of the number of function evaluations NFE_{tp} is set to 10^6. We also investigate the influence of the parameters on the performance of SinSLSO in following subsection. The parameters settings of the compared algorithms are used the recommended values as the original papers and they are given in Table 2.

5.2. Experiment results on large-scale benchmark problems

Tables 3 and 4 collect the statistical results, including the mean, standard deviation (std) and the Wilcoxon rank sum tests results, of the twelve compared algorithms on the two benchmark suites with 1000D. The Wilcoxon rank sum test results are performed under the significance level of $\alpha = 0.05$. The best result of each function among the competing algorithms is highlighted in boldface. The symbols "+" and "−" denote that SinSLSO are significantly superiority to and obviously inferior to the competing algorithm, respectively. And symbol "=" represents that the results obtained by SinSLSO and the competing algorithm are not obviously difference. Additionally, "+/=/−" in the last row of Tables means that the number of SinSLSO wins, ties and loses on the functions in total in comparing with the counterpart algorithms. It should be noted that the symbols "+" and "−" for RLLPSO, TPLSO, DLLSO, RBLSO, CCPSO2 and DECC-DG are got by directly comparing the number values for the absence of the detailed experiment results of the six algorithms. The convergence curves of compared algorithms on each function of the two benchmark suites are depicted in Figs. 6–7. Moreover, to intuitively compare the each compared algorithm, the Friedman test [67] is utilized to carried out a statistical analysis to rank compared algorithms. The average Friedman ranking of the compared algorithms and the corresponding measured p-value on these two benchmark suites are offered in Table 5.

For the CEC2010 benchmark suite, it can be observed from Table 3 that SinSLSO achieves significantly better or comparative results than the eleven competing algorithms on most of the 20 functions. SinSLSO achieves the best mean values on $F4, F6, F8, F11, F13, F16, F18$ and $F20$. DMS-L-PSO obtains the best mean values on $F9, F12, F14$ and $F17$. The number of the rest algorithms achieving the best mean values are less than 3. Specifically, compared with RLLPSO, TPLSO, DLLSO, RBLSO, CSO, SL-PSO, DMS-L-PSO, SinSLSO presents obviously advantages on 15, 12, 14, 20, 19, 18 and 15 functions, respectively. However, in comparison to these algorithms, SinSLSO only fails on 5, 8, 6, 0, 1, 0 and 5 functions, respectively. Compared with the other CCEAs (CCPSO2, DECC-DG, DECC-G, and MLCC), SinSLSO wins on 17, 17, 18, and 17 functions, and it loses only on 3, 2, 3, and 3 functions, respectively. From Fig. 6, we also can see that SinSLSO has faster convergence speed than the competing algorithms on most of the functions.

With regard to CEC2013 benchmark suite, in which functions are more tough to optimize than those in the CEC2010. We can see from Table 4 that our proposed SinSLSO still achieves better results than other algorithms on most of the 15 functions. SinSLSO obtains the best mean values on $F4 - F5, F7 - F8$, and $F11 - F12$, while the rest algorithms achieve the best mean values are less than or equal to 2 functions. In details, compared with RLLPSO, TPLSO, DLLSO, RBLSO, CSO, SL-PSO, DMS-L-PSO, SinSLSO presents obviously advantages on 10, 10, 11, 13, 12, 11 and 12 functions, respectively. Comparing with these algorithms, SinSLSO only lose the competition on 5, 5, 4, 2, 1, 1 and 3 functions, respectively. In comparison to the other CCEAs (CCPSO2, DECC-DG, DECC-G, and MLCC), SinSLSO wins on 10, 11, 10, and 10 functions, and it loses only on 5, 4, 5, and 5 functions, respectively. From Fig. 7, it also can be seen that SinSLSO outperforms the competing algorithms on most of the functions from the perspective of convergence speed.

From Table 5, the Friedman test results also evidence that our proposed SinSLSO shows the superior performance than the other compared algorithms. SinSLSO achieves the best ranking on these two benchmark suites, followed by TPLSO and RLLPSO. The fourth performing algorithms are DLLSO on the CEC2010 test suite and RBLSO on the CEC2013 test suite. These Friedman test results are consistent with the above conclusions.

In a word, from Tables 3–5, we can draw the conclusion that SinSLSO performs better or is highly competitive against the eleven compared algorithms on most functions of the two benchmark suites. The main reasons for the superiority of SinSLSO are given as follows. First, it adopts the sinusoidal function to improve exploration and exploitation capabilities of proposed algorithm. Second, the new learning strategy with random parameter having powerful search ability can assist it to jump out of the local optima and explore more promising areas. Third,

Table 3
Statistical results between SinSLSO and other algorithms on CEC2010 test functions with 1000*D*.

Method NO.	SinSLSO Mean/Std	RLLPSO Mean/Std	TPLSO Mean/Std	DLLSO Mean/Std	RBLSO Mean/Std	CSO Mean/Std
*F*1	1.16E−21/1.14E−22	4.48E−22/1.70E−22 (−)	2.93E−19/1.81E−20 (+)	3.13E−22/8.03E−23 (−)	1.55E−20/6.27E−22 (+)	3.05E−17/1.25E−18 (+)
*F*2	5.74E+02/1.82E+01	9.72E+02/5.71E+01 (+)	6.17E+02/3.10E+01 (+)	9.82E+02/4.39E+01 (+)	9.82E+02/4.65E+01 (+)	6.29E+02/2.34E+01 (+)
*F*3	5.67E−14/5.09E−15	8.67E−14/1.08E−14 (+)	9.04E−14/2.15E−15 (+)	**2.76E−14/2.38E−15 (−)**	9.91E−14/3.39E−14 (+)	1.19E−12/2.49E−14 (+)
*F*4	**3.15E+11/6.15E+10**	6.50E+11/5.02E+10 (+)	3.63E+11/7.11E+10 (+)	4.40E+11/1.10E+11 (+)	7.40E+11/1.08E+11 (+)	1.24E+12/1.92E+11 (+)
*F*5	1.14E+07/3.28E+06	1.26E+07/2.64E+06 (+)	9.25E+06/2.36E+06 (−)	1.22E+07/3.43E+06 (+)	2.32E+08/9.89E+07 (+)	**3.74E+06/1.33E+06 (−)**
*F*6	**3.55E−09/7.81E−13**	3.35E−01/5.17E−01 (+)	1.71E−07/9.00E−09 (+)	5.20E−01/7.46E−01 (+)	8.43E−09/8.04E−10 (+)	8.02E−07/1.38E−08 (+)
*F*7	1.03E−01/9.06E−02	2.37E−01/1.35E−01 (+)	**2.60E−04/1.71E−04 (−)**	7.19E+02/2.59E+03 (+)	1.37E+03/5.46E+02 (+)	1.05E+04/2.95E+03 (+)
*F*8	**8.90E+05/3.88E+06**	1.43E+07/1.13E+07 (+)	3.01E+06/2.60E+05 (+)	2.34E+07/2.46E+05 (+)	2.67E+07/3.61E+05 (+)	4.13E+07/1.16E+07 (+)
*F*9	2.16E+07/1.82E+06	3.00E+07/1.13E+06 (+)	2.07E+07/1.12E+06 (−)	4.36E+09/4.28E+06 (+)	3.61E+07/2.98E+06 (+)	6.20E+07/4.50E+06 (+)
*F*10	7.04E+02/4.22E+01	9.00E+02/5.79E+01 (+)	**5.37E+02/2.93E+01 (−)**	8.91E+02/3.66E+01 (+)	1.00E+03/6.83E+01 (+)	9.61E+03/8.57E+01 (+)
*F*11	**2.06E−13/9.64E−15**	2.41E+00/3.11E+00 (+)	1.25E−12/4.53E−14 (+)	5.80E+00/5.40E+00 (+)	5.84E−13/1.97E−13 (+)	4.31E−08/5.23E−09 (+)
*F*12	1.79E+04/1.41E+03	1.12E+04/6.09E+02 (−)	4.45E+03/3.86E+02 (−)	1.25E+04/1.46E+03 (−)	3.72E+04/1.93E+03 (+)	5.04E+05/6.09E+04 (+)
*F*13	**5.59E+02/1.89E+02**	9.62E+02/1.44E+02 (+)	6.19E+02/1.04E+02 (+)	7.35E+02/1.93E+02 (+)	1.05E+03/5.09E+02 (+)	9.31E+02/3.31E+02 (+)
*F*14	7.26E+07/4.30E+06	1.10E+09/2.32E+06 (+)	6.55E+07/2.78E+06 (−)	1.24E+08/7.38E+06 (+)	1.38E+08/9.68E+06 (+)	2.84E+08/2.04E+07 (+)
*F*15	8.78E+02/6.55E+01	8.41E+02/4.71E+01 (−)	9.60E+03/6.15E+01 (+)	**8.33E+02/4.31E+01 (−)**	1.03E+04/8.50E+01 (+)	1.01E+04/7.46E+01 (+)
*F*16	**3.06E−13/1.17E−14**	1.88E+00/3.02E+00 (+)	1.75E−12/3.21E−13 (+)	4.25E+00/2.41E+00 (+)	8.94E−13/8.48E−13 (+)	5.83E−08/6.94E−09 (+)
*F*17	1.70E+05/1.23E+04	5.31E+04/1.63E+03 (−)	5.38E+04/2.08E+03 (−)	9.05E+04/3.53E+03 (−)	1.77E+05/7.86E+03 (+)	2.17E+06/1.00E+05 (+)
*F*18	**1.10E+03/1.93E+02**	2.10E+03/3.90E+02 (+)	1.39E+03/2.57E+02 (+)	2.55E+03/8.32E+02 (+)	1.93E+03/6.17E+02 (+)	4.78E+03/3.70E+03 (+)
*F*19	5.15E+06/3.15E+05	8.71E+05/2.31E+04 (−)	1.19E+06/3.91E+04 (−)	1.80E+06/9.96E+04 (−)	7.44E+06/4.02E+05 (+)	9.69E+06/7.50E+05 (+)
*F*20	**9.47E+02/1.22E+01**	1.84E+03/1.68E+02 (+)	1.31E+03/7.19E+01 (+)	1.88E+03/1.90E+02 (+)	1.34E+03/1.24E+02 (+)	1.01E+03/4.00E+01 (+)
+/=/−	−/−/−	15/0/5	12/0/8	14/0/6	20/0/0	19/0/1
Method NO.	SL-PSO Mean/Std	DMS-L-PSO Mean/Std	CCPSO2 Mean/Std	DECC-DG Mean/Std	DECC-G Mean/Std	MLCC Mean/Std
*F*1	2.20E−18/6.01E−20 (+)	1.49E+07/1.19E+06 (+)	2.96E+00/6.68E+00 (+)	5.04E+04/2.26E+05 (+)	1.06E−13/7.43E−14 (+)	**4.38E−23/1.94E−22 (−)**
*F*2	5.80E+02/2.89E+01 (=)	3.76E+03/1.56E+02 (+)	4.30E+00/1.11E+00 (−)	4.38E+03/1.55E+02 (+)	1.30E+02/2.59E+01 (−)	**2.30E−07/1.40E−07 (−)**
*F*3	3.49E−13/4.60E−15 (+)	1.22E+01/9.02E−02 (+)	4.51E−03/1.66E−03 (+)	1.68E+01/2.02E−01 (+)	1.94E+00/1.67E−01 (+)	4.99E+00/6.80E+00 (+)
*F*4	6.94E+11/8.98E+10 (+)	4.54E+11/9.78E+10 (+)	1.69E+12/1.02E+12 (+)	3.60E+12/9.78E+11 (+)	1.31E+13/2.81E+12 (+)	1.07E+13/3.28E+12 (+)
*F*5	1.11E+07/2.91E+06 (=)	1.15E+08/2.31E+07 (+)	4.16E+08/1.37E+08 (+)	9.07E+07/1.83E+07 (+)	2.74E+08/7.66E+07 (+)	4.97E+08/1.74E+08 (+)
*F*6	1.74E−07/4.87E−09 (+)	2.23E+01/1.00E+00 (+)	1.68E+07/5.20E+06 (+)	3.85E+04/2.07E+05 (+)	5.31E+06/1.14E+06 (+)	1.96E+07/2.21E+06 (+)
*F*7	7.85E+01/4.79E+01 (+)	3.42E+06/1.08E+05 (+)	2.05E+08/2.03E+08 (+)	2.53E+04/1.30E+04 (+)	4.69E+06/5.64E+06 (+)	2.85E+06/6.10E+06 (+)
*F*8	2.55E+07/3.23E+06 (+)	2.04E+07/2.73E+06 (+)	4.13E+07/3.82E+07 (+)	4.63E+07/2.64E+07 (+)	7.08E+07/3.11E+07 (+)	9.30E+08/3.26E+09 (+)
*F*9	2.55E+07/2.02E+06 (+)	**1.91E+07/1.31E+06 (−)**	1.01E+08/3.28E+07 (+)	5.43E+07/3.52E+06 (+)	2.35E+08/2.35E+07 (+)	1.30E+08/1.65E+07 (+)
*F*10	8.73E+03/7.30E+01 (+)	4.41E+03/2.17E+02 (+)	5.02E+O3/7.80E+02 (+)	4.23E+03/1.33E+02 (+)	9.29E+03/1.50E+03 (+)	1.18E+04/4.21E+03 (+)
*F*11	3.42E−12/5.82E−14 (+)	1.30E+02/2.83E+00 (+)	1.96E+02/2.11E+00 (+)	1.29E+01/5.78E−01 (+)	2.54E+01/2.22E+00 (+)	2.02E+02/1.11E+01 (+)
*F*12	1.89E+05/2.50E+04 (+)	**4.54E+01/1.22E+01 (−)**	3.35E+04/1.15E+04 (+)	1.58E+04/2.23E+03 (−)	4.54E+04/4.90E+03 (+)	4.32E+04/7.13E+03 (+)
*F*13	9.86E+02/3.73E+02 (+)	1.40E+06/4.86E+04 (+)	1.33E+03/1.72E+02 (+)	8.60E+03/3.87E+03 (+)	3.31E+03/2.12E+03 (+)	2.48E+03/8.62E+02 (+)
*F*14	1.10E+08/5.52E+06 (+)	**1.18E+07/7.31E+05 (−)**	3.05E+08/1.15E+08 (+)	3.07E+08/1.39E+07 (+)	5.89E+08/4.82E+07 (+)	3.46E+08/2.89E+07 (+)
*F*15	1.02E+04/5.67E+01 (+)	4.91E+03/2.38E+02 (+)	1.08E+04/1.36E+03 (+)	6.07E+03/1.50E+02 (+)	8.53E+03/3.15E+03 (+)	1.74E+04/4.64E+03 (+)
*F*16	6.43E−12/7.59E−14 (+)	2.68E+02/3.11E+00 (+)	3.95E+02/5.74E−01 (+)	1.49E−01/3.82E−01 (+)	7.59E+01/1.15E+01 (+)	4.07E+02/9.92E+00 (+)
*F*17	2.02E+06/1.07E+05 (+)	**1.14E+02/2.57E+01 (−)**	1.23E+05/5.23E+04 (−)	1.26E+05/4.44E+03 (−)	1.75E+05/9.64E+03 (+)	1.85E+05/1.26E+04 (+)
*F*18	1.76E+03/4.94E+02 (+)	4.99E+04/5.87E+03 (+)	3.08E+03/2.46E+02 (+)	3.21E+09/8.81E+08 (+)	1.42E+04/8.40E+03 (+)	8.92E+03/5.26E+03 (+)
*F*19	9.74E+06/6.38E+05 (+)	1.87E+06/2.20E+05 (−)	1.52E+06/7.10E+04 (−)	2.17E+06/1.22E+05 (−)	**7.67E+05/4.08E+04 (−)**	1.55E+06/8.90E+04 (−)
*F*20	1.02E+03/4.74E+01 (+)	1.76E+03/4.36E+02 (+)	2.10E+03/1.75E+02 (+)	9.18E+10/8.99E+09 (+)	3.43E+03/2.07E+02 (+)	2.10E+03/2.19E+02 (+)
+/=/−	18/2/0	15/0/5	17/0/3	17/0/3	18/0/2	17/0/3

Table 4
Statistical results between SinSLSO and other algorithms on CEC2013 test functions with 1000*D*.

Method NO.	SinSLSO Mean/Std	RLLPSO Mean/Std	TPLSO Mean/Std	DLLSO Mean/Std	RBLSO Mean/Std	CSO Mean/Std
*F*1	1.35E−21/1.04E−22	9.01E−22/3.50E−22 (−)	3.13E−19/1.39E−20 (+)	3.99E−22/1.32E−22 (−)	1.58E−20/4.36E−22 (+)	3.62E−17/1.16E−18 (+)
*F*2	7.76E+02/2.89E+01	1.04E+03/7.40E+01 (+)	7.17E+02/2.52E+01 (−)	1.14E+03/5.78E+01 (+)	9.94E+02/4.49E+01 (+)	7.17E+02/2.54E+01 (−)
*F*3	2.16E+01/7.31E−03	2.16E+01/6.21E−03 (+)	2.16E+01/5.20E−03 (+)	2.16E+01/4.07E−03 (+)	2.16E+01/5.05E−03 (+)	2.16E+01/6.83E−03 (+)
*F*4	**3.18E+09/5.59E+08**	4.44E+09/4.99E+08 (+)	5.57E+09/8.76E+08 (+)	6.68E+09/1.68E+09 (+)	9.23E+09/1.28E+09 (+)	1.19E+10/2.39E+09 (+)
*F*5	**5.25E+05/6.25E+04**	7.25E+05/1.01E+05 (+)	6.71E+05/8.95E+04 (+)	7.00E+05/1.28E+05 (+)	5.93E+05/6.62E+04 (+)	8.08E+05/1.47E+05 (+)
*F*6	1.06E+06/1.05E+03	1.06E+06/9.95E+02 (−)	1.06E+06/9.27E+02 (−)	1.06E+06/8.28E+02 (−)	1.06E+06/1.22E+03 (−)	1.06E+06/1.09E+03 (=)
*F*7	**1.57E+05/2.00E+05**	9.84E+05/2.47E+05 (+)	2.95E+05/1.38E+05 (+)	1.60E+06/8.38E+05 (+)	7.14E+06/1.87E+06 (+)	8.04E+06/3.10E+06 (+)
*F*8	**5.91E+13/1.68E+13**	8.56E+13/1.07E+13 (+)	1.33E+14/2.10E+13 (+)	1.20E+14/3.35E+13 (+)	1.32E+14/1.42E+13 (+)	3.02E+14/6.95E+13 (+)
*F*9	**3.32E+07/6.64E+06**	3.85E+07/8.14E+06 (+)	3.98E+07/5.35E+06 (+)	1 30E+08/3.97E+07 (+)	3.42E+07/3.86E+06 (+)	4.28E+07/8.07E+06 (+)
*F*10	9.41E+07/2.21E+05	9.40E+07/2.15E+05 (−)	9.39E+07/1.78E+05 (−)	9.40E+07/2.11E+05 (−)	9.39E+07/1.64E+05 (−)	9.40E+07/2.09E+05 (=)
*F*11	**7.68E+07/4.26E+07**	9.25E+11/8.70E+09 (+)	1.29E+08/3.88E+07 (+)	9.30E+11/9.50E+09 (+)	1.05E+09/3.51E+08 (+)	4.80E+08/5.79E+08 (+)
*F*12	**1.01E+03/2.41E+01**	1.87E+03/1.72E+02 (+)	1.12E+03/8.13E+01 (+)	1.79E+03/1.39E+02 (+)	1.38E+03/7.90E+01 (+)	1.34E+03/1.24E+02 (+)
*F*13	1.04E+08/3.78E+07	2.17E+08/5.46E+07 (+)	**9.07E+07/3.55E+07 (−)**	3.35E+08/1.71E+08 (+)	1.07E+09/4.18E+08 (+)	8.48E+08/3.45E+08 (+)
*F*14	1.29E+08/5.49E+07	5.06E+07/2.16E+07 (−)	**2.30E+07/4.38E+06 (−)**	1.72E+08/1.38E+08 (+)	4.93E+09/1.14E+09 (+)	3.99E+09/3.55E+09 (+)
*F*15	1.40E+07/8.61E+05	**1.63E+06/5.88E+04 (−)**	2.42E+08/1.19E+05 (+)	4.48E+06/3.32E+05 (−)	7.57E+07/5.65E+06 (+)	7.73E+07/8.02E+06 (+)
+/=/−	−/−/−	10/0/5	10/0/5	11/0/4	13/0/2	12/2/1
Method NO.	SL-PSO Mean/Std	DMS-L-PSO Mean/Std	CCPSO2 Mean/Std	DECC-DG Mean/Std	DECC-G Mean/Std	MLCC Mean/Std
*F*1	2.23E−18/6.90E−20 (+)	1.13E+09/1.08E+08 (+)	4.10E+01/3.13E+01 (+)	1.29E+05/5.44E+05 (+)	2.10E−11/1.49E−11 (+)	**1.13E−23/3.27E−23 (−)**
*F*2	8.45E+02/4.65E+02 (−)	7.44E+03/1.99E+03 (+)	**3.60E+01/4.85E+00 (−)**	1.54E+04/7.55E+02 (+)	9.05E+01/2.36E+01 (−)	1.05E+03/8.56E+02 (+)
*F*3	2.16E+01/7.81E−03 (=)	2.10E+01/2.13E−01 (−)	**2.00E+01/1.27E−04 (−)**	2.08E+01/1.02E−02 (−)	2.01E+01/2.13E−03 (−)	2.00E+01/4.83E−02 (−)
*F*4	7.10E+09/1.52E+09 (+)	3.06E+11/4.85E+10 (+)	3.50E+10/2.17E+10 (+)	1.79E+11/5.88E+10 (+)	9.41E+10/4.15E+10 (+)	9.86E+10/4.17E+10 (+)
*F*5	7.64E+05/1.62E+05 (+)	3.73E+06/4.47E+05 (+)	1.40E+07/4.81E+06 (+)	5.34E+06/4.43E+05 (+)	8.15E+06/1.61E+06 (+)	1.94E+07/8.63E+06 (+)
*F*6	1.06E+06/1.12E+03 (=)	**1.01E+06/1.00E+04 (−)**	1.05E+06/5.25E+03 (−)	1.06E+06/1.41E+03 (−)	1.06E+06/1.92E+03 (−)	1.05E+06/9.09E+03 (−)
*F*7	3.11E+06/1.46E+06 (+)	1.29E+09/5.75E+08 (+)	4.13E+08/9.37E+08 (+)	1.35E+09/3.66E+08 (+)	4.39E+08/3.18E+08 (+)	6.53E+08/5.45E+08 (+)
*F*8	1.34E+14/2.72E+13 (+)	1.56E+14/5.54E+13 (+)	1.18E+15/9.98E+14 (+)	7.79E+15/2.08E+15 (+)	3.03E+15/1.32E+15 (+)	5.15E+15/2.54E+15 (+)
*F*9	4.35E+07/7.12E+06 (+)	3.67E+08/6.80E+07 (+)	3.76E+09/1.01E+09 (+)	1.27E+09/1.75E+08 (+)	6.49E+08/1.88E+08 (+)	1.17E+09/5.49E+08 (+)
*F*10	9.40E+07/2.05E+05 (=)	**9.19E+07/1.13E+06 (−)**	9.30E+07/7.02E+05 (−)	9.38E+07/3.45E+05 (−)	9.29E+07/6.71E+05 (−)	9.35E+07/9.93E+05 (−)
*F*11	2.21E+09/6.41E+09 (+)	6.86E+10/3.50E+10 (+)	9.36E+11/1.54E+10 (+)	9.21E+11/3.55E+09 (+)	3.71E+10/2.13E+10 (+)	8.51E+10/6.97E+10 (+)
*F*12	1.13E+03/1.34E+02 (+)	2.36E+07/1.51E+07 (+)	2.11E+03/1.77E+02 (+)	8.76E+10/1.04E+10 (+)	3.37E+03/2.41E+02 (+)	2.20E+03/2.26E+02 (+)
*F*13	6.96E+08/2.60E+08 (+)	1.85E+10/1.20E+10 (+)	4.03E+09/2.30E+09 (+)	2.38E+10/5.72E+09 (+)	6.15E+09/1.86E+09 (+)	7.52E+09/2.57E+09 (+)
*F*14	7.04E+09/6.68E+09 (+)	2.30E+11/1.12E+11 (+)	9.10E+10/8.53E+10 (+)	1.16E+11/3.49E+10 (+)	8.96E+10/3.95E+10 (+)	1.26E+11/5.42E+10 (+)
*F*15	5.61E+07/6.51E+06 (+)	1.61E+07/2.93E+06 (+)	4.76E+06/5.08E+06 (−)	1.37E+07/2.11E+06 (−)	5.10E+06/3.35E+05 (−)	7.55E+06/1.00E+06 (−)
+/=/−	11/3/1	12/0/3	10/0/5	11/0/4	10/0/5	10/0/5

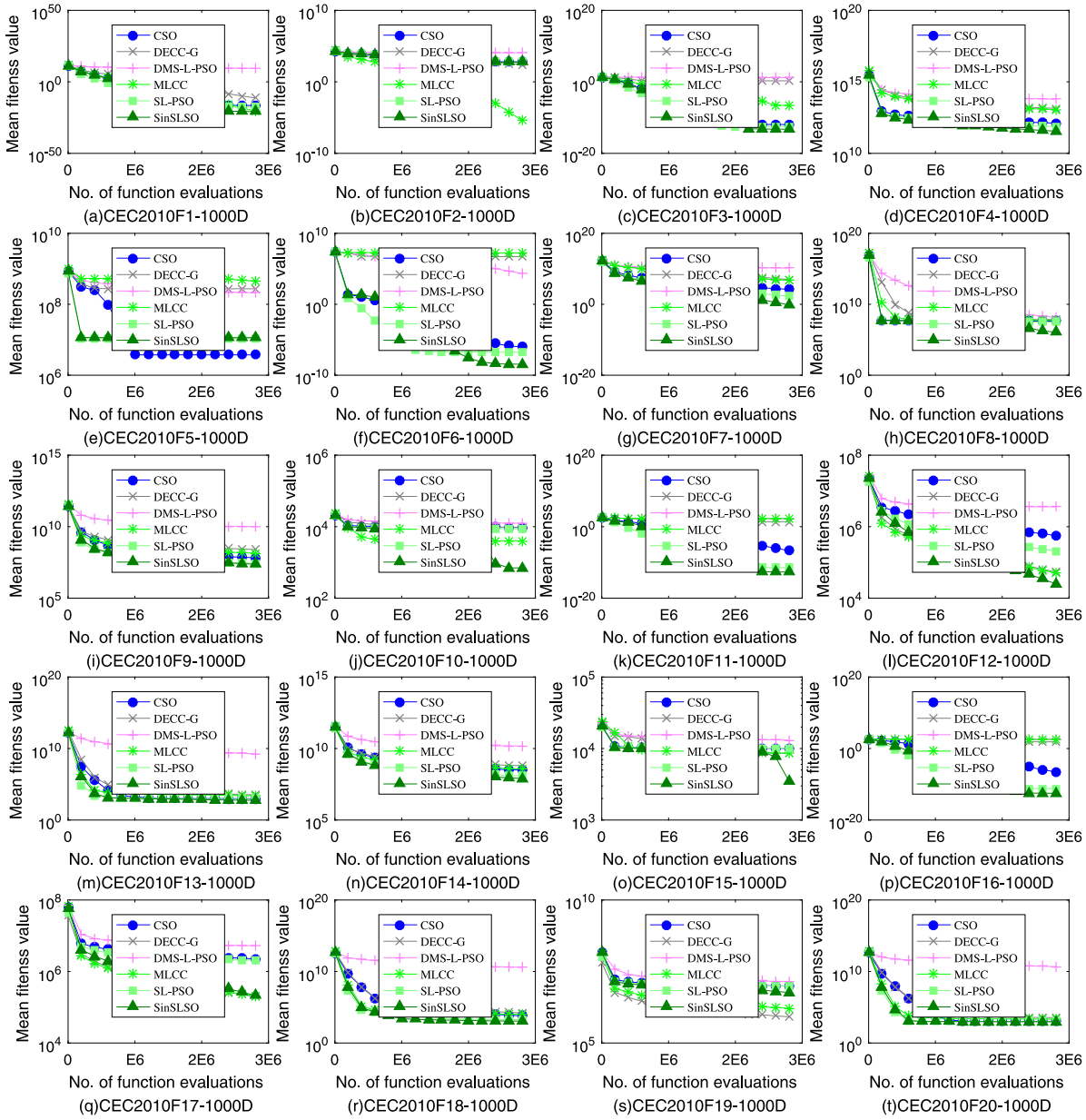

Fig. 6. Convergence curves of SinSLSO on CEC2010 benchmark functions with 1000*D*.

Table 5
Average ranking of each competing algorithm obtained by the Friedman test.

Algorithms	SinSLSO	RLLPSO	TPLSO	DLLSO	RBLSO	CSO	SL-PSO	DMS-L-PSO	CCPSO2	DECC-DG	DECC-G	MLCC	p-value
Ranking (CEC2010)	2.85	4.85	3	5.225	6.225	7.225	6.05	6.95	8.5	8.7	9.05	9.375	2.54E−13
Rank	1	3	2	4	6	8	5	7	9	10	11	12	
Ranking (CEC2013)	3.8	5.0333333	4.533333	6.166667	5.833333	6.733333	6.433333	8.266667	7	9.533333	7.133333	7.533333	3.96E−04
Rank	1	3	2	5	4	7	6	11	8	12	9	10	

the population size reduction mechanism can preserve the population diversity at the beginning of evolution and improve the convergence speed at the ending of evolution. In short, these three strategies make SinSLSO obtain good performance.

5.3. Parameter analysis

Parameters have a considerable influence on the performance of the meta-heuristic algorithm [68]. Similar to the CSO, DLLSO and TPLSO, the performance of our proposed SinSLSO mainly depends on the population size *ps* and control parameter φ. In general, a small population size tend to lead the algorithm to converge prematurely. Conversely, a large population size tend to degrade algorithm performance because more function evaluations are required for each generation. According to CSO [22], it can be found that there exist a correlation between *ps* and φ. Therefore, we carry out experiments to investigate the effect of *ps* and φ on SinSLSO. In our experiment, the *ps* varies from 400 to 800 and the φ varies from 0.15 to 0.35.

Table 6 lists the statistical results of SinSLSO having differential combinations of these two parameters *ps* and φ on seven CEC2010 functions $F1, F3, F6, F9, F10, F15$, and $F20$, where $F1$

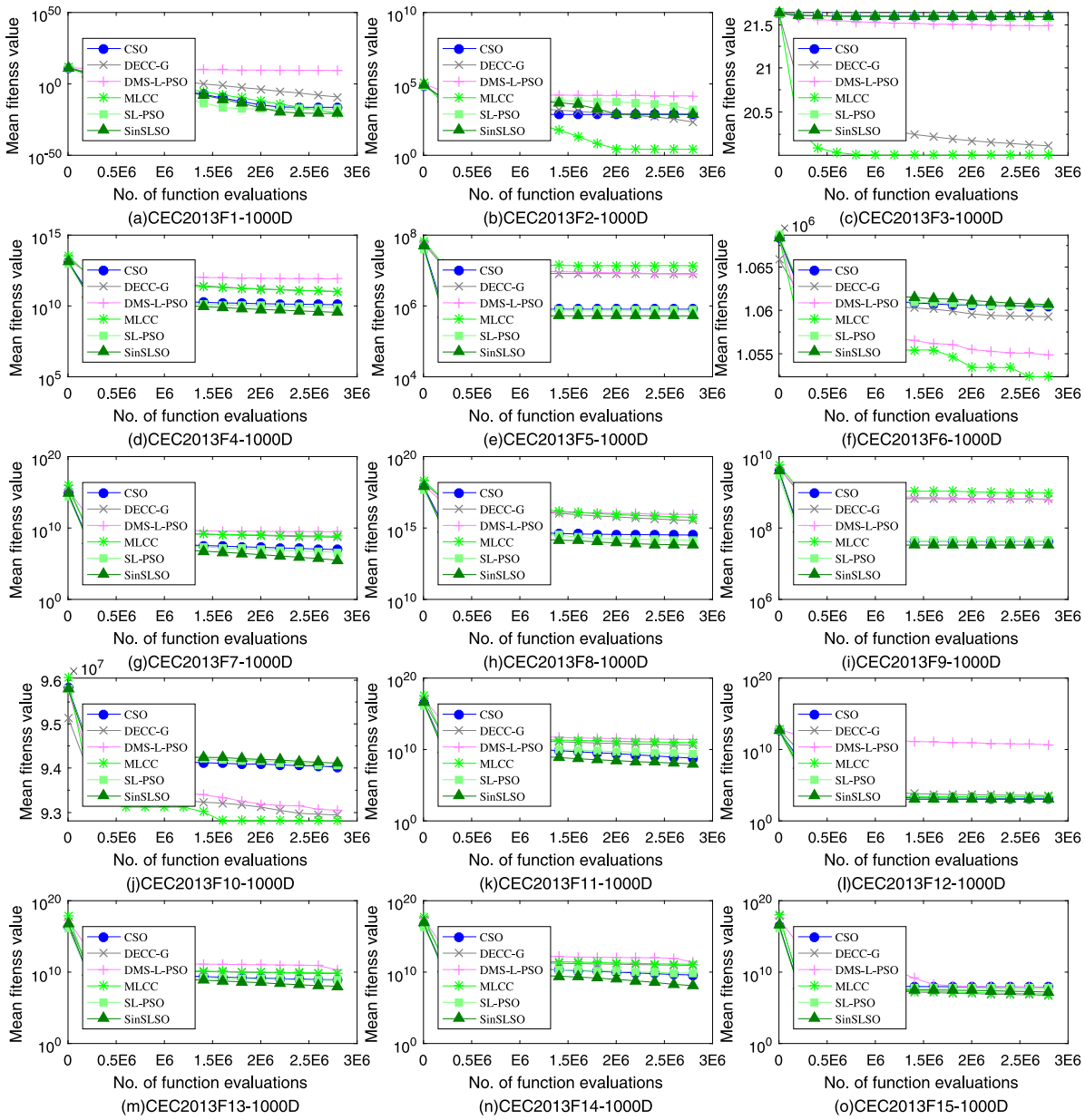

Fig. 7. Convergence curves of SinSLSO on CEC2013 benchmark functions with 1000*D*.

and $F3$ are separable functions, $F20$ is fully non-separable and the remaining functions are partially separable. In additionally, $F1$ and $F9$ are unimodal functions, while the rest ones are multimodal functions. Although these functions are selected, they consist of almost all types of test problems: fully separable, partially separable, fully non-separable, unimodal and multimodal. In general, partially separable and fully non-separable functions are more tough to solve than fully separable ones, multimodal functions are more tough to solve in comparison with unimodal ones. Most of real-world optimization problems are multimodal and inseparable, so we should pay more attention these types of functions.

From Table 6, it can be seen that SinSLSO with different parameter values has different performance. SinSLSO with $ps = 400$ and $\varphi = 0.3$ is good at unimodal functions $F1$ and $F9$. SinSLSO with $ps = 400$ and $\varphi = 0.35$ performs better on $F15$. SinSLSO with $ps = 600$ and $\varphi = 0.25$ has better performance on $F10$. SinSLSO with $ps = 800$ and $\varphi = 0.25$ obtains better performance on $F16$. SinSLSO with $ps = 800$ and $\varphi = 0.3$ achieves better performance on $F3$ and $F20$. We also can find that SinSLSO with $ps = 800$ has best mean results on 3 functions, which is equal to the SinSLSO with $ps = 400$. SinSLSO with $\varphi = 0.3$ obtains best mean results on 4 functions, with $\varphi = 0.25$ gets best mean results on 2 functions, with $\varphi = 0.35$ has best mean result on only 1 function, while $\varphi = 0.15$ and $\varphi = 0.2$ do not have any best mean results. Based on the above observations, in our proposed SinSLSO algorithm, we adopt a population size reduction mechanism with ($ps_{\max} = 800$, $ps_{\min} = 400$) to balance the diversity and convergence, and set the parameter φ in a random way with ($\varphi_{\max} = 0.3$, $\varphi_{\min} = 0.25$).

5.4. Effect of the sinusoidal function and the population reduction strategy

To investigate the effect of the sinusoidal function and the population reduction strategy on SinSLSO, we conduct comparison experiments on 1000*D* CEC 2010 benchmark functions with four variants of the proposed SinSLSO algorithm, i.e., RandSLSO

Table 6
Mean values of optimization errors obtained by SinSLSO on several functions in CEC 2010 test functions of 1000D with the population size ps varying from 400 to 800, and φ from 0.15 to 0.35.

	$ps = 400$					$ps = 600$					$ps = 800$				
NO.	$\varphi = 0.15$	$\varphi = 0.2$	$\varphi = 0.25$	$\varphi = 0.3$	$\varphi = 0.35$	$\varphi = 0.15$	$\varphi = 0.2$	$\varphi = 0.25$	$\varphi = 0.3$	$\varphi = 0.35$	$\varphi = 0.15$	$\varphi = 0.2$	$\varphi = 0.25$	$\varphi = 0.3$	$\varphi = 0.35$
$F1$	1.63E−21	8.45E−22	5.74E−22	**4.45E−22**	5.43E−22	3.79E−21	1.61E−21	9.75E−22	8.96E−22	1.23E−21	3.73E−18	3.71E−21	1.99E−21	1.98E−21	4.72E−21
$F3$	7.83E−14	8.72E−14	7.87E−14	7.12E−14	6.15E−14	5.26E−14	5.26E−14	5.36E−14	4.88E−14	4.73E−14	1.99E−12	5.13E−14	4.62E−14	**4.44E−14**	5.04E−14
$F6$	1.84E+01	2.03E+01	1.99E+01	2.00E+01	2.05E+01	2.63E−01	2.09E−01	2.81E−01	1.29E−01	5.65E−02	1.71E−08	4.14E−09	**3.61E−09**	3.64E−09	4.68E−09
$F9$	2.35E+07	1.95E+07	1.87E+07	**1.75E+07**	1.83E+07	3.03E+07	2.39E+07	2.22E+07	2.09E+07	2.22E+07	3.47E+07	2.82E+07	2.49E+07	2.43E+07	2.53E+07
$F10$	1.26E+03	1.07E+03	1.07E+03	1.03E+03	1.03E+03	8.69E+03	8.67E+02	**7.21E+02**	7.31E+02	3.41E+03	9.34E+03	7.92E+03	5.48E+03	5.98E+03	8.24E+03
$F15$	7.71E+03	1.15E+03	1.20E+03	1.18E+03	**1.12E+03**	1.04E+04	1.02E+04	1.92E+03	1.15E+03	1.01E+04	1.03E+04	1.01E+04	9.97E+03	9.99E+03	1.01E+04
$F20$	1.21E+03	1.28E+03	1.26E+03	1.16E+03	1.01E+03	9.88E+02	9.75E+02	9.82E+02	9.58E+02	9.62E+02	9.72E+02	9.72E+02	9.61E+02	**9.54E+02**	9.64E+02

Table 7
Mean and Std of optimization errors obtained by five SinSLSO variants on CEC 2010 test functions with 1000*D*.

Algorithm NO	SinSLSO Mean/Std	SinSLSO ($ps = 200$) Mean/Std	SinSLSO ($ps = 500$) Mean/Std	SinSLSO ($ps = 800$) Mean/Std	RandSLSO Mean/Std
$F1$	1.16E−21/1.14E−22	3.31E+04/4.78E+04 (+)	6.59E−22/1.31E−22 (−)	1.81E−21/1.77E−22 (+)	1.33E−21/1.02E−22 (+)
$F2$	5.74E+02/1.82E+01	2.21E+03/1.04E+02 (+)	8.35E+02/4.02E+01 (+)	5.57E+02/2.78E+01 (−)	5.46E+02/2.32E+01 (−)
$F3$	5.67E−14/5.09E−15	3.32E+00/2.98E−01 (+)	5.93E−14/5.42E−15 (=)	4.51E−14/3.28E−15 (−)	6.31E−14/3.02E−15 (+)
$F4$	3.15E+11/6.15E+10	2.16E+11/9.03E+10 (−)	2.68E+11/5.58E+10 (−)	3.94E+11/7.74E+10 (+)	2.06E+11/3.40E+10 (−)
$F5$	1.14E+07/3.28E+06	3.24E+07/7.81E+06 (+)	1.49E+07/3.88E+06 (+)	1.01E+07/3.05E+06 (=)	1.02E+07/2.65E+06 (=)
$F6$	3.55E−09/7.81E−13	1.99E+01/1.53E−02 (+)	9.38E+00/8.90E+00 (+)	3.67E−09/8.43E−11 (+)	1.20E+01/8.01E+00 (+)
$F7$	1.03E−01/9.06E−02	5.34E+05/9.26E+04 (+)	2.39E−02/8.21E−02 (−)	3.33E+00/2.08E+00 (+)	3.52E−02/2.24E−02 (−)
$F8$	8.90E+05/3.88E+06	2.25E+07/2.40E+07 (+)	1.42E+04/2.05E+04 (−)	3.56E+05/4.84E+04 (+)	3.00E+04/2.53E+04 (=)
$F9$	2.16E+07/1.82E+06	3.36E+07/5.51E+06 (+)	1.94E+07/1.77E+06 (−)	2.48E+07/2.50E+06 (+)	1.86E+07/1.51E+06 (−)
$F10$	7.04E+02/4.22E+01	2.29E+03/1.21E+02 (+)	8.44E+02/4.16E+01 (+)	5.08E+03/3.84E+02 (+)	6.58E+02/3.69E+01 (−)
$F11$	2.06E−13/9.64E−15	6.10E+01/9.30E+00 (+)	5.86E+00/7.64E+00 (+)	1.90E−13/5.09E−15 (−)	4.71E−02/2.10E−01 (+)
$F12$	1.79E+04/1.41E+03	4.59E+04/1.11E+04 (+)	6.07E+03/7.00E+02 (−)	3.89E+04/3.42E+03 (+)	1.94E+04/2.01E+03 (+)
$F13$	5.59E+02/1.89E+02	2.30E+04/2.34E+04 (+)	6.45E+02/2.06E+02 (=)	5.56E+02/1.38E+02 (=)	5.59E+02/1.28E+02 (=)
$F14$	7.26E+07/4.30E+06	1.10E+08/1.11E+07 (+)	6.30E+07/3.72E+06 (−)	8.95E+07/6.83E+06 (+)	6.40E+07/4.16E+06 (−)
$F15$	8.78E+02/6.55E+01	2.46E+03/1.35E+02 (+)	8.99E+02/4.22E+01 (=)	1.00E+04/5.00E+01 (+)	9.66E+02/2.90E+02 (=)
$F16$	3.06E−13/1.17E−14	1.34E+02/2.00E+01 (+)	3.47E−01/6.31E−01 (=)	3.01E−13/1.01E−14 (=)	3.12E−13/1.25E−14 (=)
$F17$	1.70E+05/1.23E+04	1.27E+05/1.53E+04 (−)	7.96E+04/5.54E+03 (−)	3.22E+05/1.35E+04 (+)	1.91E+05/9.37E+03 (+)
$F18$	1.10E+03/1.93E+02	5.78E+03/2.75E+03 (+)	1.24E+03/2.39E+02 (+)	1.17E+03/2.29E+02 (=)	1.26E+03/2.57E+02 (+)
$F19$	5.15E+06/3.15E+05	1.35E+06/6.96E+04 (−)	3.72E+06/1.86E+05 (−)	8.17E+06/4.69E+05 (+)	5.35E+06/3.08E+05 (+)
$F20$	9.47E+02/1.22E+01	3.10E+03/2.40E+02 (+)	1.02E+03/6.89E+01 (+)	9.61E+02/2.00E+01 (+)	9.36E+02/9.33E−01 (−)
+/=/−	−/−/−	17/0/3	7/4/9	13/4/3	8/5/7
Mean ranking	2.4	4.4	2.55	3.1	2.55
Rank	1	5	2	4	2

with the learning probability set with *rand* function, SinSLSO ($ps = 200$) with a fixed population size $ps = 200$, SinSLSO ($ps = 500$) with a fixed population size $ps = 500$, and SinSLSO ($ps = 800$) with a fixed population size $ps = 800$. The parameter settings of RandSLSO are the same as those of SinSLSO except for the learning probability. The parameter settings of SinSLSO ($ps = 200$), SinSLSO ($ps = 500$), and SinSLSO ($ps = 800$) are the same as those of SinSLSO except for the population size. Each variant is independently run 20 times to obtain the experiment results, which are listed in Table 7. The Wilcoxon rank sum test and the Friedman test are utilized to make comparison among the compared algorithms. From the third-to-last row of Table 7, i.e., the results of the Wilcoxon rank sum test, we can see that SinSLSO significantly outperforms SinSLSO ($ps = 200$) and SinSLSO ($ps = 800$). SinSLSO ($ps = 500$) has slightly better performance than SinSLSO, but SinSLSO ($ps = 500$) has poor performance on F16, which may be that SinSLSO ($ps = 500$) is trapped in a local optimum. Fortunately, the population reduction strategy of our proposed SinSLSO algorithm can avoid this problem. These results demonstrate the effectiveness of the population reduction strategy. We can also see that SinSLSO performs a little better than RandSLSO, which shows that the sinusoidal function has an effect on SinSLSO. From the last row of Table 7, i.e., the results of the Friedman test, it can be observed that our proposed SinSLSO has best mean ranking which indicates that SinSLSO has best performance, as well as the sinusoidal function and the population reduction strategy have effects on SinSLSO.

5.5. Exploration and exploitation investigation for SinSLSO

Exploration and exploitation are two important aspects of meta-heuristic algorithm. When addressing unimodal problems, meta-heuristic algorithm should appropriately emphasize exploitation to try to get fast convergence. On the other hand, when dealing with multimodal problems, meta-heuristic algorithm should properly emphasize exploration to jump out of the local optimum [69]. However, exploration and exploitation are usually in conflict, so a meta-heuristic algorithm should make a good trade-off between these two aspects. To demonstrate that SinSLSO can properly compromise these two aspects, a group of experiments are carried out to compare SinSLSO with GPSO [14] and CSO on four CEC2010 functions ($F1$ is fully separable and unimodal, $F7$ is partially separable and unimodal, $F11$ and $F13$ are partially separable and multimodal) in regard to fitness value as well as population diversity.

The diversity used in this paper is defined as follows [70].

$$D(\boldsymbol{P}) = \frac{1}{ps}\sum_{i=1}^{ps}\sqrt{\sum_{d=1}^{D}\left(x_i^d - \bar{x}^d\right)^2} \tag{35}$$

$$\bar{x}^d = \frac{1}{ps}\sum_{i=1}^{ps} x_i^d \tag{36}$$

where $D(\boldsymbol{P})$ denotes the diversity of the population $\boldsymbol{P}$, $\bar{x}^d$ is the mean of the dth dimension of all particles in the population. The compared results of these three algorithms are plotted in Fig. 8. The maximum number of FEs is set to $3000 \times D$, the population size is set to 500 for GPSO and CSO, the maximum and minimum population size for SinSLSO are set to 800 and 400, respectively.

From Fig. 8, at the beginning of evolution process, three compared algorithms have high diversity since algorithms need to explore the search space at this stage. During the search progress, the GPSO reserves the highest population diversity but achieves the worst performance due to the stagnation of the population. We can also see that the proposed SinSLSO has faster convergence than the GPSO and CSO. For SinSLSO and CSO, the population diversity gradually decreases as the number of iterations increases, which is consistent with the convergence of algorithms. Especially for $F13$, SinSLSO can enhance population diversity in the middle of the evolution process. SinSLSO performs better than GPSO and CSO because it can make a good balance between exploration and exploitation abilities.

6. The proposed SinSLSO for the feature selection problem

In this section, we apply the proposed SinSLSO algorithm to deal with the feature selection problem. In last few decades, feature selection has become an emerging field since it can improve the predictive accuracy of algorithms and reduce computation cost required for data mining [71]. Feature selection is an critical step utilized in several tasks, such as image classification, data mining, pattern recognition, and cluster analysis. The purpose of

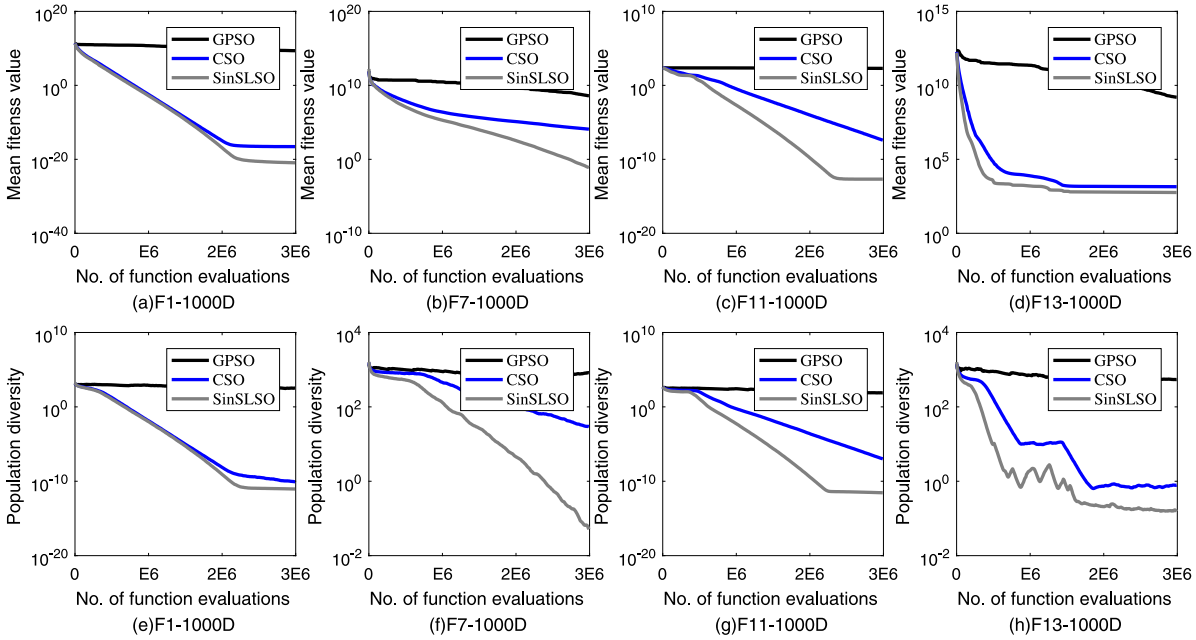

Fig. 8. Fitness values and population diversity of GPSO, CSO, and SinSLSO on four CEC2010 functions with 1000D, namely, $F1$, $F7$, $F11$ and $F13$.

feature selection to choose the most useful features that contribute to the constructed model more efficiently and effectively, and eliminate noisy, irrelevant or redundant features. However, finding useful subsets of features is a challenging task due to the large search space and complex interaction among features [72]. Given a dataset having n features, the number of all possible feature subsets is as large as 2^n, which makes it very unlikely to search all possible subsets for a large n since it is too costly and restrictive. Hence, feature selection is an NP-hard combinatorial optimization problem. Due to the inefficiency of traditional search approaches in solving this combinatorial problem, some metaheuristic algorithms have been adopted to solve this combinatorial problem [73,74]. In this paper, we apply SinSLSO to deal with the feature selection problem to further examine the performance of SinSLSO in handling the real-world problem.

6.1. Fitness function

The aim of feature selection is to find a feature subset with a high classification capability. In other words, it is to minimize the error rate of classification. Therefore, feature selection can be modeled as the following minimization problem.

$$\begin{aligned} &\min f(x) \\ &s.t. x \in \chi \end{aligned} \tag{37}$$

where x denotes a solution containing a set of selected features and $\chi \in \Re^N$ represents the feasible solution set. In this study, we adopt the k nearest neighbor (kNN) with $K = 5$ as a classifier to evaluate feature subsets which are represented by the particles. To improve the reliability of performance evaluation, we use the average error rates of ten-fold cross-validation on training data as the fitness value.

6.2. Individual encoding

In this study, the numerical encoding of particle is used to deal with the feature selection problem. Each particle denotes the total number of features of a given dataset and each variable indicates the probability to select the corresponding feature. Taking a dataset having D features as an example, a particle with D variables is defined as follows.

$$X_i = (x_{i,1}, x_{i,2}, \ldots, x_{i,D}), i = 1, 2, \ldots, ps \tag{38}$$

where $x_{i,j} \in [0, 1]$ represents the probability of the jth feature be selected. Each particle is decoded to binary string with a predefined threshold η. The jth feature is selected and the related variable is decoded to 1 if and only if its value is greater than η. Otherwise, the jth feature is discarded and the corresponding variable is decoded to 0. In this paper, the η is set to 0.6 according to [74] to limit the number of selected features.

Table 8
Description of five data sets.

Dataset	Number of features	Number of patterns	Number of class
movement	90	360	15
arrhythmia	279	452	16
madelon	500	2600	2
isolet5	617	1559	26
InterAd	1588	3279	2

6.3. Experiments analysis for feature selection

In this section, the performance of the SinSLSO for feature selection problem is verified on several high-dimensional real-world data sets from different knowledge fields, including movement, arrhythmia, madelon, isolet5, and InterAd. These data sets are cited from UCI machine learning repository [75]. The description of these data sets are listed in Table 8. The experimental results of our proposed SinSLSO are compared with experimental results reported in [73]. The compared algorithms include CSO, PSO, and four variants of PSO proposed in [74]. For the sake of fairness, the maximum number of fitness evaluations is set to 10000, which is the same as [73]. During the period of population size reduction, the maximum and minimum number of

Table 9
Average error rates on five UCI data sets.

Dataset	SinSLSO	CSO-kNN	PSO-kNN	Xue1-kNN	Xue2-kNN	Xue3-kNN	Xue4-kNN
movement	**0.1354/0.0071**	0.2394/0.0326 (+)	0.2850/0.0392 (+)	0.2850/0.0399 (+)	0.2896/0.0415 (+)	0.2832/0.0409 (+)	0.2869/0.0327 (+)
arrhythmia	0.3249/0.1802	**0.3240/0.0225 (−)**	0.4059/0.0221 (+)	0.4076/0.0203 (+)	0.3593/0.0335 (+)	0.4098/0.0243 (+)	0.3564/0.0294 (+)
madelon	0.2094/0.0139	**0.1572/0.0374 (−)**	0.4111/0.0189 (+)	0.4066/0.0210 (+)	0.2704/0.1006 (+)	0.4168/0.0230 (+)	0.3736/0.1027 (+)
isolet5	**0.0654/0.0020**	0.1491/0.0124 (+)	0.1875/0.0118 (+)	0.1855/0.0141 (+)	0.1917/0.0157 (+)	0.1922/0.0172 (+)	0.1954/0.0149 (+)
InterAd	**0.0167/0.0008**	0.0289/0.0044 (+)	0.0404/0.0071 (+)	0.0406/0.0058 (+)	0.0403/0.0055 (+)	0.0411/0.0056 (+)	0.0401/0.0050 (+)
+/=/−	−/−/−	3/0/2	5/0/0	5/0/0	5/0/0	5/0/0	5/0/0

population size ps are set to 200 and 100, respectively. The other parameters of SinSLSO are the same as Section 5.1.

The average error rates on five UCI data sets are shown in Table 9 and the best results on each data set are shown in boldface. From Table 9, we can observe that our proposed SinSLSO performs competitively among the seven comparison algorithms. Specifically, our proposed SinSLSO performs best on three data sets movement, isolet5 and InterAd. The CSO performs best on two data sets arrhythmia and madelon. These experimental results demonstrate that SinSLSO is competitive to deal with feature selection problem.

7. Conclusions

An efficient sinusoidal social learning swarm optimizer (SinSLSO) is introduced in this study. The proposed SinSLSO adopts the sinusoidal function to compromise the exploration and exploitation capabilities. Besides, to balance between the diversity and convergence, SinSLSO employs the trapezoidal function to decrease the population size. Furthermore, a novel learning strategy with random parameters is designed which can help the SinSLSO escape from local optimum. To demonstrate the efficiency of the proposed SinSLSO, which is compared with eleven recently proposed and state-of-the-art algorithms on two widely used benchmark suites. Experiment results confirm that SinSLSO can obtain better results than the compared algorithms on most of large-scale test problems. In addition, the proposed SinSLSO is applied to deal with the feature selection problem and results on feature selection also verify the competitive performance of the proposed SinSLSO.

SinSLSO shows good performance in handling large-scale problems, but some of its solutions are still far from the global optimum. This problem is also common for the compared algorithms as given in Tables 3 and 4. Hence, how to further enhance SinSLSO to achieve solutions as close to the global optimum as possible is our future work.

CRediT authorship contribution statement

Nengxian Liu: Conceptualization, Investigation, Methodology, Software, Data curation, Visualization, Writing – original draft. **Jeng-Shyang Pan:** Supervision, Investigation, Writing – review & editing. **Shu-Chuan Chu:** Writing – review & editing, Validation. **Pei Hu:** Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

[1] S. Mahdavi, M.E. Shiri, S. Rahnamayan, Metaheuristics in large-scale global continues optimization: A survey, Inform. Sci. 295 (2015) 407–428.
[2] J.R. Jian, Z.H. Zhan, J. Zhang, Large-scale evolutionary optimization: A survey and experimental comparative study, Int. J. Mach. Learn. Cybern. 11 (3) (2020) 729–745.
[3] Z.J. Wang, Z.H. Zhan, W.J. Yu, Y. Lin, J. Zhang, T.L. Gu, J. Zhang, Dynamic Group Learning Distributed Particle Swarm Optimization for Large-Scale Optimization and Its Application in Cloud Workflow Scheduling, IEEE Trans. Cybern. 50 (6) (2020) 2715–2729.
[4] N. Liu, J.S. Pan, T.T. Nguyen, A bi-population QUasi-Affine TRansformation Evolution algorithm for global optimization and its application to dynamic deployment in wireless sensor networks, EURASIP J. Wirel. Commun. Netw. 2019 (1) (2019).
[5] S.Y. Ho, H.S. Lin, W.H. Liauh, S.J. Ho, OPSO: Orthogonal particle swarm optimization and its application to task assignment problems, IEEE Trans. Syst. Man Cybern. A 38 (2) (2008) 288–298.
[6] H. Shokri-Ghaleh, A. Alfi, S. Ebadollahi, A. Mohammad Shahri, S. Ranjbaran, Unequal limit cuckoo optimization algorithm applied for optimal design of nonlinear field calibration problem of a triaxial accelerometer, Measurement 164 (2020).
[7] N. Liu, J.S. Pan, J. Wang, T.T. Nguyen, An adaptation multi-group quasi-affine transformation evolutionary algorithm for global optimization and its application in node localization in wireless sensor networks, Sensors 19 (19) (2019).
[8] P. Hu, J.S. Pan, S.C. Chu, Improved Binary Grey Wolf Optimizer and Its application for feature selection, Knowl.-Based Syst. 195 (2020).
[9] K. Price, R. Storn, Differential Evolution — A Simple and Efficient Heuristic for Global Optimization over Continuous Spaces, J. Global Optim. (11) (1997) 341–359.
[10] S. Das, S.S. Mullick, P.N. Suganthan, Recent advances in differential evolution — An updated survey, Swarm Evol. Comput. 27 (2016) 1–30.
[11] J.S. Pan, N. Liu, S.C. Chu, A Hybrid Differential Evolution Algorithm and Its Application in Unmanned Combat Aerial Vehicle Path Planning, IEEE Access 8 (2020) 17691–17712.
[12] Z. Meng, J.S. Pan, Monkey King Evolution: A new memetic evolutionary algorithm and its application in vehicle fuel consumption optimization, Knowl.-Based Syst. 97 (2016) 144–157.
[13] J.S. Pan, Z. Meng, S.C. Chu, H.R. Xu, Monkey King Evolution: An enhanced ebb-tide-fish algorithm for global optimization and its application in vehicle navigation under wireless sensor network environment, Telecommun. Syst. 65 (3) (2017) 351–364.
[14] J. Kennedy, R. Eberhart, Particle swarm Optimization, in: Proceedings of IEEE International Conference on Neural Networks, B T - Icnn95-International Conference on Neural Networks, IEEE, Perth, WA, 1995, pp. 1942–1948.
[15] Z. Meng, J.S. Pan, H. Xu, QUasi-Affine TRansformation Evolutionary (QUATRE) algorithm: A cooperative swarm based algorithm for global optimization, Knowl.-Based Syst. 109 (2016) 104–121.
[16] J.-S. Pan, P. Hu, S.-C. Chu, Binary Fish Migration Optimization for Solving Unit Commitment, Energy (2021) 120329, http://dx.doi.org/10.1016/j.energy.2021.120329.
[17] D. Karaboga, B. Basturk, A powerful and efficient algorithm for numerical function optimization: Artificial bee colony (ABC) algorithm, J. Global Optim. 39 (3) (2007) 459–471.
[18] R. Cheng, Y. Jin, A social learning particle swarm optimization algorithm for scalable optimization, Inform. Sci. 291 (2015) 43–60.
[19] Q. Yang, W.N. Chen, J.D. Deng, Y. Li, T. Gu, J. Zhang, A Level-Based Learning Swarm Optimizer for Large-Scale Optimization, IEEE Trans. Evol. Comput. 22 (4) (2018) 578–594.
[20] H. Wang, M. Liang, C. Sun, G. Zhang, L. Xie, Multiple-strategy learning particle swarm optimization for large-scale optimization problems, Complex Intell. Syst. (2020).
[21] R. Lan, Y. Zhu, H. Lu, Z. Liu, X. Luo, A Two-Phase Learning-Based Swarm Optimizer for Large-Scale Optimization, IEEE Trans. Cybern. (2020) 1–10.
[22] R. Cheng, Y. Jin, A competitive swarm optimizer for large scale optimization, IEEE Trans. Cybern. 45 (2) (2015) 191–204.

[23] W.N. Chen, J. Zhang, Y. Lin, N. Chen, Z.H. Zhan, H.S.H. Chung, Y. Li, Y.H. Shi, Particle swarm optimization with an aging leader and challengers, IEEE Trans. Evol. Comput. 17 (2) (2013) 241–258.
[24] J.J. Liang, A.K. Qin, P.N. Suganthan, S. Baskar, Comprehensive learning particle swarm optimizer for global optimization of multimodal functions, IEEE Trans. Evol. Comput. 10 (3) (2006) 281–295.
[25] N. Lynn, P.N. Suganthan, Heterogeneous comprehensive learning particle swarm optimization with enhanced exploration and exploitation, Swarm Evol. Comput. 24 (2015) 11–24.
[26] X. Chen, H. Tianfield, C. Mei, W. Du, G. Liu, Biogeography-based learning particle swarm optimization, Soft Comput. 21 (24) (2017) 7519–7541.
[27] Z.H. Zhan, J. Zhang, Y. Li, Y.H. Shi, Orthogonal learning particle swarm optimization, IEEE Trans. Evol. Comput. 15 (6) (2011) 832–847.
[28] D.B. Chen, C.X. Zhao, Particle swarm optimization with adaptive population size and its application, Appl. Soft Comput. 9 (1) (2009) 39–48.
[29] C. Sun, J. Zeng, J. Pan, S. Xue, Y. Jin, A new fitness estimation strategy for particle swarm optimization, Inform. Sci. 221 (2013) 355–370.
[30] R. Cheng, C. Sun, Y. Jin, A multi-swarm evolutionary framework based on a feedback mechanism, in: 2013 IEEE Congress on Evolutionary Computation, CEC 2013, 2013, pp. 718–724.
[31] S.Z. Zhao, J.J. Liang, P.N. Suganthan, M.F. Tasgetiren, Dynamic multi-swarm particle swarm optimizer with local search for large scale global optimization, in: 2008 IEEE Congress on Evolutionary Computation, CEC 2008, 2008, pp. 3845–3852.
[32] H. Deng, L. Peng, H. Zhang, B. Yang, Z. Chen, Ranking-based biased learning swarm optimizer for large-scale optimization, Inform. Sci. 493 (2019) 120–137.
[33] H. Li, J. Yu, M. Yang, F. Kong, Secure Outsourcing of Large-Scale Convex Optimization Problem in Internet of Things, IEEE Internet Things J. 9 (11) (2022) 8737–8748.
[34] C. Wang, Y. Wang, Y. Cheung, A branch and bound irredundant graph algorithm for large-scale MLCS problems, Pattern Recognit. 119 (2021).
[35] M. Xiao, J. Zhang, K. Cai, X. Cao, T. Ke, Cooperative co-evolution with weighted random grouping for large-scale Crossing Waypoints Locating in Air Route Network, in: Proceedings - International Conference on Tools with Artificial Intelligence, ICTAI, 2011, pp. 215–222.
[36] X. Chen, A. Shen, Self-adaptive differential evolution with Gaussian—Cauchy mutation for large-scale CHP economic dispatch?problem, Neural Comput. Appl. (2022).
[37] Y. Dongsheng, W. Mingliang, L. Di, X. Yunlang, Z. Xianyu, Y. Zhile, Dynamic opposite learning enhanced dragonfly algorithm for solving large-scale flexible job shop scheduling problem, Knowl.-Based Syst. 238 (2022) 107815.
[38] H. Liu, B. Wu, A. Maleki, F. Pourfayaz, An improved particle swarm optimization for optimal configuration of standalone photovoltaic scheme components, Energy Sci. Eng. 10 (3) (2022) 772–789.
[39] N. Ayyash, F. Hejazi, Development of hybrid optimization algorithm for structures furnished with seismic damper devices using the particle swarm optimization method and gravitational search algorithm, Earthq. Eng. Eng. Vib. 21 (2) (2022) 455–474.
[40] R. Rashid, N. Nambath, Area Optimisation of Two Stage Miller Compensated Op-Amp in 65 nm Using Hybrid PSO, IEEE Trans. Circuits Syst. II 69 (1) (2022) 199–203.
[41] M. Sato, Y. Fukuyama, T. Iizaka, T. Matsui, Total Optimization of Energy Networks in a Smart City by Multi-Swarm Differential Evolutionary Particle Swarm Optimization, IEEE Trans. Sustain. Energy 10 (4) (2019) 2186–2200.
[42] Z. Li, Y. Chen, Y. Song, K. Lu, J. Shen, Effective Covering Array Generation Using an Improved Particle Swarm Optimization, IEEE Trans. Reliab. 71 (1) (2022) 284–294.
[43] C. Pozna, R.E. Precup, E. Horvath, E.M. Petriu, Hybrid Particle Filter-Particle Swarm Optimization Algorithm and Application to Fuzzy Controlled Servo Systems, IEEE Trans. Fuzzy Syst. (2022) http://dx.doi.org/10.1109/TFUZZ.2022.3146986.
[44] H. Lin, C. Tang, Analysis and Optimization of Urban Public Transport Lines Based on Multiobjective Adaptive Particle Swarm Optimization, IEEE Trans. Intell. Transp. Syst. (2021) http://dx.doi.org/10.1109/TITS.2021.3086808.
[45] Y. Luo, X. Ouyang, J. Liu, L. Cao, Y. Zou, An image encryption scheme based on particle swarm optimization algorithm and hyperchaotic system, Soft Comput. 26 (11) (2022) 5409–5435.
[46] T.H. Van, S. Tangaramvong, S. Limkatanyu, H.N. Xuan, Two-phase ESO and comprehensive learning PSO method for structural optimization with discrete steel sections, Adv. Eng. Softw. 167 (2022).
[47] D. Zhu, Z. Huang, L. Xie, C. Zhou, Improved Particle Swarm Based on Elastic Collision for DNA Coding Optimization Design, IEEE Access (2022) http://dx.doi.org/10.1109/ACCESS.2022.3150275.
[48] J.C. Seck-Tuoh-Mora, J. Medina-Marin, E.S. Martinez-Gomez, E.S. Hernandez-Gress, N. Hernandez-Romero, V. Volpi-Leon, Cellular particle swarm optimization with a simple adaptive local search strategy for the permutation flow shop scheduling problem, Arch. Control Sci. 29 (2) (2019) 205–226.
[49] F.V.D. Bergh, A.P. Engelbrecht, A cooperative approach to particle swarm optimization, IEEE Trans. Evol. Comput. 8 (3) (2004) 225–239.
[50] X. Li, X. Yao, Cooperatively coevolving particle swarms for large scale optimization, IEEE Trans. Evol. Comput. 16 (2) (2012) 210–224.
[51] Y.J. Shi, H.F. Teng, Z.Q. Li, Cooperative co-evolutionary differential evolution for function optimization, Lecture Notes in Comput. Sci. 3611 (PART II) (2005) 1080–1088.
[52] Z. Yang, K. Tang, X. Yao, Large scale evolutionary optimization using cooperative coevolution, Inform. Sci. 178 (15) (2008) 2985–2999.
[53] Z. Yang, K. Tang, X. Yao, Multilevel cooperative coevolution for large scale optimization, in: 2008 IEEE Congress on Evolutionary Computation, CEC 2008, 2008, pp. 1663–1670.
[54] M.N. Omidvar, X. Li, Y. Mei, X. Yao, Cooperative co-evolution with differential grouping for large scale optimization, IEEE Trans. Evol. Comput. 18 (3) (2014) 378–393.
[55] Y. Sun, M. Kirley, S.K. Halgamuge, Extended differential grouping for large scale global optimization with direct and indirect variable interactions, in: GECCO 2015 - Proceedings of the 2015 Genetic and Evolutionary Computation Conference, 2015, pp. 313–320.
[56] Y. Mei, M.N. Omidvar, X. Li, X. Yao, A competitive divide-and-conquer algorithm for unconstrained large-scale black-box optimization, ACM Trans. Math. Software 42 (2) (2016) 1–24.
[57] M.N. Omidvar, M. Yang, Y. Mei, X. Li, X. Yao, DG2: A faster and more accurate differential grouping for large-scale black-box optimization, IEEE Trans. Evol. Comput. 21 (6) (2017) 929–942.
[58] Q. Yang, W.N. Chen, T. Gu, H. Zhang, J.D. Deng, Y. Li, J. Zhang, Segment-Based Predominant Learning Swarm Optimizer for Large-Scale Optimization, IEEE Trans. Cybern. 47 (9) (2017) 2896–2910.
[59] R. Tanabe, A.S. Fukunaga, Improving the search performance of SHADE using linear population size reduction, in: Proceedings of the 2014 IEEE Congress on Evolutionary Computation, CEC 2014, 2014, pp. 1658–1665.
[60] S. Das, A. Konar, U.K.B.T. Chakraborty, Two improved differential evolution schemes for faster global search, in: Genetic and Evolutionary Computation Conference, ACM, Washington D.C., USA, 2005, pp. 991–998.
[61] N. Liu, J.-S. Pan, S.-C. Chu, A Competitive Learning QUasi Affine TRansformation Evolutionary for Global Optimization and Its Application in CVRP, J. Internet Technol. 21 (7) (2020) 1863–1883.
[62] M. Clerc, J. Kennedy, The particle swarm-explosion, stability, and convergence in a multidimensional complex space, IEEE Trans. Evol. Comput. 6 (1) (2002) 58–73.
[63] I.C. Trelea, The Particle Swarm Optimization Algorithm: Convergence Analysis and Parameter Selection, Inform. Process. Lett. 85 (6) (2003) 317–325.
[64] K. Tang, X. Li, P.N. Suganthan, Z. Yang, T. Weise, Benchmark Functions for the CEC'2010 Special Session and Competition on Large-Scale Global Optimization, Tech. Rep., (33) University of Science and Technology of China. China, Hefei, China, 2010, pp. 1–23.
[65] X. Li, K. Tang, M. Omidvar, Z. Yang, K. Qin, Benchmark Functions for the CEC'2013 Special Session and Competition on Large-Scale Global Optimization, Tech. Rep., (33) RMIT University, Melbourne, Australia, 2013, p. 8.
[66] F. Wang, X. Wang, S. Sun, A reinforcement learning level-based particle swarm optimization algorithm for large-scale optimization, Inform. Sci. 602 (2022) 298–312.
[67] J. Carrasco, S. García, M.M. Rueda, S. Das, F. Herrera, Recent trends in the use of statistical tests for comparing swarm and evolutionary computing algorithms: Practical guidelines and a critical review, Swarm Evol. Comput. 54 (2020) arXiv:2002.09227.
[68] J.S. Pan, N. Liu, S.C. Chu, T. Lai, An efficient surrogate-assisted hybrid optimization algorithm for expensive optimization problems, Inform. Sci. 561 (2021) 304–325.
[69] Y. Volkan Pehlivanoglu, A new particle swarm optimization method enhanced with a periodic mutation strategy and neural networks, IEEE Trans. Evol. Comput. 17 (3) (2013) 436–452.
[70] O. Olorunda, A.P. Engelbrecht, Measuring exploration/exploitation in particle swarms using swarm diversity, in: 2008 IEEE Congress on Evolutionary Computation, CEC 2008, 2008, pp. 1128–1134.
[71] Y. Zhang, D. Gong, Y. Hu, W. Zhang, Feature selection algorithm based on bare bones particle swarm optimization, Neurocomputing 148 (2015) 150–157.
[72] E. Hancer, B. Xue, M. Zhang, D. Karaboga, B. Akay, Pareto front feature selection based on artificial bee colony optimization, Inform. Sci. 422 (2018) 462–479, http://dx.doi.org/10.1016/j.ins.2017.09.028.
[73] S. Gu, R. Cheng, Y. Jin, Feature selection for high-dimensional classification using a competitive swarm optimizer, Soft Comput. 22 (3) (2018) 811–822.
[74] B. Xue, M. Zhang, W.N. Browne, Particle swarm optimization for feature selection in classification: A multi-objective approach, IEEE Trans. Cybern. 43 (6) (2013) 1656–1671.
[75] A. Frank, A. Asuncion, UCI Machine Learning Repository, 2010, URL http://archive.ics.uci.edu/ml.